



Monitoring the Health and Reliability of Remote Autonomous Systems

Submitted as Honours Dissertation in SIT724

SUBMISSION DATE

September 2026

Daniel Mattioli

STUDENT ID: s223377562

Bachelor of Software Engineering - Honours (S464)

Supervised by: Kevin Lee

Abstract

Remote autonomous systems (RAS), such as unmanned aerial vehicles and ground robots, are becoming more common in environments where failures are costly and human intervention is not always possible (Puchalski & Giernacki 2022). However, existing system health monitoring solutions are mainly reactive and tailored for static systems that rely on cloud infrastructure instead of mobile platforms that have limited resources and unreliable connectivity (Gültekin et al. 2022). There is currently a lack of practical and lightweight frameworks for real-time RAS monitoring that do not depend on cloud infrastructure or pre-labelled fault data.

This thesis designs, implements and evaluates a Prometheus-based monitoring framework for RAS across two studies. Research Question 1 (RQ1) asks whether such a framework can assess system health, and Research Question 2 (RQ2) asks whether it can detect faults without pre-labelled data.

For RQ1, a pipeline of a custom Python telemetry exporter, Prometheus and Grafana was evaluated on the Drone Telemetry Tampering Dataset v2 (Ekanayakadevilk 2024), a public dataset of DJI flight records with six labelled anomaly phases. It detected every fault phase transition with no false positives during normal operation and met three of four evaluation criteria in full. Detection latency was rated partial because of the Prometheus pull-based scrape interval (Barrett et al. 2023).

For RQ2, the same pipeline was connected to a physical DJI Tello drone, and four alerting rules were derived from five baseline flights using the aircraft's own measured data. The rules detected nine of nine induced faults across battery, thermal and link classes, at a median latency of 6.0 s, with no sustained false alert in 13.2 min of held-out normal flight. Five of six evaluation criteria were met in full.

Together, the studies show that collecting and displaying telemetry is solved by existing open-source tooling, but calibrating what that telemetry means is not. A derived threshold is only valid within the conditions it was measured in, and should be reported with the scope of those conditions.

Contents

Abstract	i
1 Introduction	1
1.1 Research Questions	2
1.2 Contributions	2
1.3 Thesis Structure	3
2 Literature Review	4
2.1 Introduction	4
2.2 System Health and Reliability in Remote Autonomous Systems	4
2.3 Monitoring in Remote Autonomous Systems	6
2.4 Anomaly Detection using Machine Learning	9
2.5 Threshold-Based and Adaptive Fault Detection	10
2.6 Self-Healing in Autonomous Systems	13
2.7 Monitoring Frameworks and Observability	15
2.8 Research Gap	17
3 Methodology	19
3.1 Overview	19
3.1.1 Candidate Methods and Selection	20
3.1.2 Experimental Platform and Tooling	21
3.2 Aims	22
3.3 Research Questions	22
3.4 RQ1 Methodology	23
3.5 RQ2 Methodology	25
3.5.1 Platform Telemetry Constraints	25
3.5.2 Data Collection Protocol	27
3.5.3 Threshold Derivation Procedure	29
3.5.4 Data Analysis Plan	29
3.6 Timeline and Milestones	31
3.7 Ethical Considerations	34
3.7.1 Human Participants and Ethics Determination	34
3.7.2 Flight Safety and Regulatory Compliance	34
3.7.3 Data Management and Privacy	35
3.7.4 Research Integrity and Use of Generative AI	35
3.8 Threats to Results	36
3.9 Summary	37
4 Prometheus-Based Health Monitoring Framework for Remote Autonomous Systems (RQ1)	39
4.1 Aim	39
4.2 Implementation	39
4.2.1 Python Telemetry Exporter	39
4.2.2 Prometheus	41
4.2.3 Grafana	42

4.3	Experimental Setup	42
4.3.1	Dataset	42
4.3.2	Software Configuration	42
4.3.3	Dataset Phases	43
4.4	Results	43
4.4.1	Prometheus Metric Collection	43
4.4.2	Grafana Dashboard	44
4.4.3	PromQL Queries Used	47
4.4.4	Evaluation Criteria Results	48
4.5	Analysis	49
4.6	Answer to RQ1	50
5	Autonomous Threshold-Based Fault Detection on a Physical Remote Autonomous System (RQ2)	51
5.1	Aim	51
5.2	Design	52
5.2.1	Monitoring Architecture	52
5.2.2	The Calibration Procedure	53
5.3	Implementation	55
5.3.1	Session Harness	55
5.3.2	Fault Injection Mechanisms	56
5.3.3	Metric Set and Platform Constraints	57
5.3.4	Alerting Rules	57
5.3.5	Analysis Tooling	57
5.4	Experimental Setup	58
5.4.1	Platform and Environment	58
5.4.2	Session Protocol	59
5.4.3	Experimental Runs	60
5.5	Phase 1 Results: Baseline Characterisation (RQ2.1)	60
5.5.1	Session Summary	60
5.5.2	Platform Behaviour	61
5.5.3	Derived Thresholds	63
5.5.4	Assumed and Derived Thresholds Compared	65
5.6	Phase 2 Results: Fault Detection (RQ2.2)	65
5.6.1	Held-Out Normal Flights	65
5.6.2	Battery Depletion	66
5.6.3	Thermal Stress	68
5.6.4	Link Degradation	69
5.6.5	Detection Outcomes	70
5.7	Phase 3 Results: Evaluation (RQ2.3)	72
5.7.1	Detection Latency	72
5.7.2	False Positive Rate	73
5.7.3	Evaluation Criteria Results	74
5.8	Analysis	76
5.9	Answer to RQ2	77

6	Discussion	79
7	Conclusion	81
7.1	Future Work	82

List of Figures

1	Prometheus Target health page confirming the RAS exporter endpoint is actively scraped with status UP.	41
2	Prometheus browser showing <code>ras_health_status</code> transitioning between 1 (healthy) and 0 (fault) across dataset phases.	45
3	Grafana dashboard displaying real-time RAS telemetry replayed from the DJI flight dataset. Fault phase transitions are visible as step changes in the health status panel.	46
4	Architecture of the Study 2 monitoring system. The live telemetry path runs from the aircraft to the dashboard during a flight. The calibration loop runs offline between flights and decides what the alerting rules contain.	52
5	The threshold calibration procedure applied throughout Study 2	54
6	The test environment during a live session, with the Tello airborne and the harness running on the ground station. Depth of field has been added in post-processing to separate the aircraft from the background.	59
7	FAULT-BAT-03 depleting through the derived 31 % threshold. The alert fires 6 s after the crossing and leaves 122.4 s before the firmware failsafe engages at 9 %. 67	
8	FAULT-THERM-03 against the healthy control WARM-01. Raw 1 Hz samples are shown faintly with an 11-sample rolling mean over them, because the field is reported in whole degrees. The fault run launches at 90 °C and cools towards the normal operating band, while the control never reaches the derived ceiling. .	69
9	How detection latency is measured. Only the interval from the telemetry-derived onset to the Prometheus alert is counted.	72
10	The Study 2 Grafana dashboard during FAULT-LINK-01, showing all four rule-bearing metrics, their derived thresholds and the live alert state on one screen.	74

List of Tables

1	Comparison of Candidate Research Methods	20
2	Required Resources	21
3	Research Design Matrix	24
4	Study 1 Evaluation Criteria (RQ1)	25
5	Tello SDK Telemetry Fields Used in Study 2	26
6	Threshold Derivation Rules by Metric Class	30
7	Study 2 Evaluation Criteria (RQ2)	32
8	Research Milestones	32
9	Risks and Mitigations	33
10	RAS Telemetry Metrics Exposed to Prometheus	41
11	Dataset Phases Replayed Through the Monitoring Pipeline	43
12	Evaluation Criteria Results	48
13	Study 2 Experimental Runs	60
14	Phase 1 Baseline Sessions	61
15	Onboard Temperature by Elapsed Flight Time (°C), Mean per Interval	61
16	Thresholds Derived from Phase 1 Baseline Data	63
17	Assumed and Derived Thresholds Compared	65
18	Battery Depletion Runs	66
19	Thermal Runs and the Healthy Control	68
20	Link Degradation Runs	69
21	Detection Outcomes Across the Nine Fault Runs	70

22	Unintended Sustained Alerts Raised During the Fault Runs	71
23	Detection Latency by Fault Class	72
24	Study 2 Evaluation Criteria Results	75

1 Introduction

Remote autonomous systems (RAS) are increasingly being deployed across a wide range of environments where direct human oversight is limited or unavailable (Hemalatha et al. 2025). Drones are used for infrastructure inspection, search and rescue operations, and environmental monitoring (Puchalski & Giernacki 2022). Underwater vehicles conduct seabed surveys and pipeline inspections. Ground-based robots operate in disaster zones, warehouses, and remote agricultural settings (Singh et al. 2025). What these systems share is that they operate far from the people responsible for maintaining them, and they are expected to function reliably without continuous human intervention.

The consequence of failure in these environments is significant. When a remote autonomous system breaks down in the field, the cost is not just the hardware. It is the lost mission, the delayed response, the inability to reach the system to diagnose the problem (Gültekin et al. 2022). In safety-critical applications such as emergency response or infrastructure monitoring, a failure that goes undetected for even a short period can have serious consequences. The further a system operates from human oversight, the higher the stakes when something goes wrong.

Despite this, most current approaches to monitoring remote autonomous systems are reactive rather than proactive (Smallman & Cook 2013). Faults are typically detected after they cause a failure, rather than identified early through continuous health observation (Yousuf et al. 2024). Existing monitoring research has largely been developed for static industrial systems or controlled laboratory environments, and there is limited evidence of how these approaches perform on physically mobile platforms operating in unpredictable conditions (Boncea & Bacivarov 2016). The result is a gap between what the literature describes and what remote autonomous systems actually need: a continuous, real-time monitoring pipeline that can track system health, detect anomalous behaviour early, and provide actionable insight without requiring a human to be watching at all times.

This thesis investigates how monitoring frameworks can be designed and evaluated for this purpose, using a DJI Tello drone as a representative remote autonomous system and Prometheus as the primary monitoring tool.

1.1 Research Questions

Two research questions guide this study:

- **RQ1:** How can monitoring frameworks be used to assess and maintain the health of remote autonomous systems?
- **RQ2:** How can system metrics such as battery level, connectivity, and performance be used to detect faults in remote autonomous systems?

RQ1 is addressed in Study 1 through the design and experimental evaluation of a Prometheus-based monitoring pipeline using publicly available drone telemetry. RQ2 is addressed in Study 2 through integration with a physical DJI Tello drone, controlled fault injection experiments, and quantitative evaluation of detection performance.

1.2 Contributions

This thesis makes the following contributions:

- A Prometheus-based monitoring framework for remote autonomous systems, including a custom Python telemetry exporter that exposes eight health metrics (altitude, speed, heading, GPS coordinates, anomaly label, health status, and fault event count) collected by Prometheus every 15 seconds.
- An evaluation of the framework on the Drone Telemetry Tampering Dataset v2 (Ekanayakadevilk 2024), which has six labelled anomaly phases. The framework detected every fault phase transition with no false positives during normal operation, meeting three of four evaluation criteria in full.
- A reusable monitoring architecture (DJI Tello SDK → Python exporter → Prometheus → Grafana) that is not tied to a single platform and can be adapted to any autonomous system that exposes numeric telemetry.

- A calibration procedure that derives alerting thresholds from a platform’s own baseline data according to how each metric behaves. On a physical DJI Tello the derived rules detected nine of nine induced faults at a median latency of 6.0 s.
- Evidence that a derived threshold is valid only within the conditions it was measured in, and should therefore be reported together with that envelope.

1.3 Thesis Structure

Section 2 reviews the relevant literature across six themes: system health and reliability, monitoring in remote autonomous systems, anomaly detection using machine learning, threshold-based and adaptive fault detection, self-healing, and monitoring frameworks and observability. Section 3 presents the research design and selected methods. Section 4 explores RQ1 through the design, implementation, and experimental evaluation of the Prometheus-based monitoring framework. Section 5 answers RQ2 by deriving alerting thresholds from a physical aircraft’s own measured baseline and testing them against induced faults. Section 6 draws the two studies together, and Section 7 concludes the thesis and identifies directions for future work.

2 Literature Review

2.1 Introduction

Remote autonomous systems are becoming increasingly advanced (Hemalatha et al. 2025), but this also raises an important challenge: how can system failures be identified before they occur? These systems operate in environments where a human cannot always step in, which means the system itself needs to be able to assess its own health and catch problems early. Monitoring sensor data continuously is how that gets done. On top of that, machine learning has opened up new ways to detect faults that do not follow a predictable pattern, and self-healing systems take it a step further by responding to problems automatically rather than waiting for someone to notice. All of these capabilities depend on one thing: a monitoring framework that gives real-time visibility into what the system is actually doing.

This review covers six themes that are at the centre of this problem: system health and reliability, monitoring of autonomous systems, machine learning-based anomaly detection, threshold-based and adaptive fault detection, self-healing, and monitoring frameworks and observability. Each theme is explored, with the key findings and limitations of current research outlined before identifying the gap this work aims to address.

2.2 System Health and Reliability in Remote Autonomous Systems

System health monitoring is the continuous tracking of operational parameters such as battery voltage, motor temperature, GPS signal quality and IMU readings to determine whether a system is functioning within safe limits Reichard et al. (2007). In remote autonomous systems, these parameters can drift gradually before any visible problem emerges, which makes early detection critical. Unlike industrial equipment with a technician nearby, a drone operating remotely has no immediate fallback when something starts to degrade. The difference between catching a fault early and missing it entirely often comes down to how well the monitoring system is set up.

Yousuf et al. (2024) showed that sensor-based monitoring can enable predictive maintenance through early detection of faults in industrial motor systems. They combine temperature, vibration and current sensors to constantly monitor the health of an AC induction motor, sending GSM alerts if the readings are outside the expected range. The significance of this approach is that the system is proactive, meaning it does not wait for a failure to occur, but detects early warning signs and raises an alert before the fault becomes significant. One limitation is that their detection logic is based on fixed thresholds, where a fault is essentially anything outside a pre-defined range. This works well in a stable industrial environment, but in a dynamic platform like a drone, where what is considered “normal” varies with flight mode, payload, and weather, fixed thresholds quickly become unreliable. The principle is still the same, but the way it is analysed has to be more flexible.

Singh et al. (2025) discuss reliability in IoT systems from a broader perspective, reviewing techniques such as fault tolerance, predictive maintenance, energy-efficient design and secure communication protocols. In their study they observe that reliability in autonomous systems is not about finding individual faults in isolation but about designing systems that can still perform well when components begin to degrade. This is especially important for remote autonomous systems. A drone or robot that is operating far from a maintenance team needs to be able to identify its own health state and report it clearly. They also mention that energy efficiency is closely related to reliability: a sensor node that runs out of battery cannot report anything, and a monitoring layer that consumes too much processing power can itself become a reliability risk. That trade-off is particularly relevant in the context of drones, where every bit of energy spent on monitoring is energy not spent on flight time.

These ideas are further developed by Gültekin et al. (2022) who look at the real-time health monitoring and fault detection of industrial autonomous transfer vehicles using edge AI. The system monitors vehicle health metrics locally and uses machine learning to detect faults without a centralised server. The edge-based approach is important as it shows that real-time health monitoring is possible even in physically mobile systems with limited or intermittent connectivity, a problem directly relevant to drone-based monitoring. Their reported results are also worth mentioning: they reduced the network bandwidth requirement by around 43 times and the total data transfer time by around 37 times by running detection at the edge as opposed to streaming everything to a cloud service. Those reductions are significant for a mobile platform where bandwidth is often the limiting factor. The tradeoff is that edge AI moves some of the processing burden onto the device itself, adding a new reliability variable that also needs to be monitored.

When analysed together, these studies show that the most successful health monitoring for

autonomous systems combines the continuous collection of sensor data with smart analysis and real-time alerts. Most of this work, however, is focused on relatively static or industrial systems, motors, factory equipment, and fixed sensor nodes, with limited application to dynamic, mobile platforms operating in unpredictable environments. Even the edge AI work in Gültekin et al., which is closest to a mobile use case, is still evaluated on transfer vehicles working in a controlled factory environment with stable network coverage. The jump from that to a drone operating outside, losing GPS, hitting wind gusts, changing flight modes, is big, and not directly covered by any of these studies.

Comparing these three studies directly, there is a clear trade-off between simplicity and adaptability. Yousuf et al.'s fixed-threshold approach is the easiest to implement and explain, but the least portable to a changing environment. Singh et al. sit at the opposite end, treating reliability as a system-wide design property rather than a specific detection technique, which is more flexible in principle but gives little concrete guidance on how to actually implement a monitoring pipeline. Gültekin et al. occupy the middle ground: a concrete, real-time, mobile-adjacent implementation, but one that still assumes stable network coverage and a controlled factory floor. None of the three is evaluated on a platform that both moves freely and operates with genuinely unreliable connectivity, which is exactly the condition a drone in the field is subject to.

Existing health monitoring research is focused primarily on industrial or stationary systems, with limited application to dynamic, mobile autonomous systems such as drones. In addition, not much is known about how health metrics can be continuously collected and acted on in real time in remote systems where connectivity can be unreliable and conditions can change rapidly. Very few studies look at how reliability monitoring should adapt when 'normal' system behaviour itself changes during operation, e.g. a drone transitioning between hovering, manoeuvring and landing.

2.3 Monitoring in Remote Autonomous Systems

In remote autonomous systems, monitoring refers to the process of collecting data continuously from sensors to assess system performance and detect faults before failures occur Barile et al. (2018). Collecting data is not enough as it has to be analysed quickly and trigger appropriate responses, in real time. As autonomous systems become more complex, and are deployed in wider ranges of environments, reliable monitoring has become a genuinely

hard problem to solve. Research in this area ranges from healthcare wearables, to industrial IoT, to aerial robotics and each of these provides a slightly different perspective on what good monitoring really looks like.

Paradeshi et al. (2022) show how to use IoT sensor data to detect faults in healthcare monitoring systems functioning in wireless environments. Their framework analyses performance data from wearable sensors, flagging irregular readings that may indicate device failure, with about 80% fault detection accuracy. Though the application is healthcare focused, the underlying approach of constantly collecting sensor data, identifying deviations from expected behaviour and raising alerts directly applies to monitoring autonomous systems in other domains. Their research gives a useful baseline to understand how monitoring frameworks can be designed and evaluated. The 80% accuracy rate is interesting, though. It might be okay for health care wearables, where a missed alert usually just means a checkup. For an autonomous drone, that same figure is one missed fault in every five, which is a big safety concern.

A more sophisticated approach is taken by Boncea & Bacivarov (2016), who suggest a system which constantly pushes metrics of IoT devices into time-series databases like Prometheus, InfluxDB and OpenTSDB. Their architecture separates the concerns of data collection, storage and analysis into different layers, allowing the monitoring system to scale up as new devices are added. Importantly, they see monitoring as a continuous process, not a one-off check. The system keeps a continuous log of system behaviour that can be queried to identify both sudden faults and gradual degradation in performance over time. This is fundamentally more powerful than threshold-based alerting alone. The architecture is also interoperable at the storage layer, so the same monitoring logic can work for many monitoring tools such as Prometheus, InfluxDB or OpenTSDB. This kind of modularity is useful for research, because it means you can swap out components without having to rebuild the whole pipeline.

More recently, UAV-oriented research has begun to explore how monitoring can be applied to mobile, airborne autonomous systems. Puchalski & Giernacki (2022) present a survey of fault detection methods for unmanned aerial vehicles, discussing techniques from model-based methods to data-driven approaches. In their review, they highlight that UAV monitoring presents unique challenges compared to ground-based systems. Flight dynamics are non-linear, sensor readings can be influenced by vibration and wind, and the consequences of undetected faults can be severe. However, most of the existing monitoring approaches are not customised for UAV platforms. Rather they adapt techniques developed for other fields. Their review also groups detection methods into wider categories like model-based, signal-based, data-driven and hybrid, and notes that the more reliable systems tend to combine more than one. This observation is important for any new UAV monitoring system, because it suggests that no single technique

alone is likely to be good enough.

Barile et al. (2018) present a fully autonomous wireless sensor network for structural and environmental monitoring, deployed to detect rockfall, landslide and structural-integrity events on bridges and protection barriers. Each node is self-configuring and energy-harvesting, meaning the network can be deployed in remote terrain without needing external power or manual reconfiguration, and continuously streams acceleration, inclination, position and temperature data to a remote server for automated alarm management. This is a genuinely autonomous, remote, real-time monitoring system, closer in spirit to a drone deployment than the industrial or healthcare examples reviewed above, and shows that reliable continuous monitoring is achievable in physically remote, resource-constrained deployments. The limitation is that the nodes themselves are static once installed; the system monitors a fixed physical structure rather than a platform that itself moves and needs to interpret its own motion as part of distinguishing a fault from a legitimate manoeuvre - a distinction that matters a great deal for a flying platform.

A common limitation in these studies is that monitoring is usually considered as a fault detection problem or a health tracking problem. Few approaches integrate both into a unified system that provides real-time health insight while supporting fault detection and diagnosis. For a remote autonomous system, both views are very useful. Knowing that the system is currently within its safe operating range is just as important as knowing when something has gone wrong. Splitting the two creates blind spots that are difficult to close after the fact.

Read together, these four studies span a spectrum from simple threshold alerting (Paradeshi et al.) through layered, tool-agnostic architecture (Boncea and Bacivarov) to fully autonomous but physically static sensor deployment (Barile et al.), with Puchalski and Giernacki's survey sitting above all three as a taxonomy of the detection techniques each could draw on. None of them, however, combine the architectural rigor of Boncea and Bacivarov with the physical autonomy of Barile et al. in a platform that also moves - which is precisely the combination a drone monitoring pipeline requires.

It is shown that current monitoring approaches are effective in detecting faults in controlled environments, but they do not provide a full picture of the system health in real-time for dynamic, mobile autonomous systems. Very little research is specifically designed for mobile platforms such as drones. Most of the existing work does not combine health tracking and fault detection in a single integrated monitoring pipeline. And there is little evidence of how these approaches perform in conditions that are routine for a drone such as intermittent connectivity, rapid changes in operating mode, and environmental disturbance, none of which are typical in the labs that most of these studies are conducted in.

2.4 Anomaly Detection using Machine Learning

Remote autonomous systems generate a lot of sensor data, and not all of it is easy to interpret with fixed rules. Traditional fault detection requires you to define what a fault looks like in advance, which is fine when the failure modes are well understood but quickly falls apart in a system like a drone where the range of possible faults is hard to predict. Machine learning changes that. Instead of asking whether a reading is above a threshold, the system learns what normal behaviour looks like and flags anything that does not fit. The shift from rule-based to learning-based detection allows anomalies to be identified based on the specific environment and behaviour of the system.

Devi et al. (2024) demonstrate how the Isolation Forest algorithm can be applied to IoT sensor data to detect anomalies effectively. Isolation Forest works by finding data points that are statistically isolated from the rest of the data set. It is best suited for finding rare or unusual events in time-series telemetry. Their system computes anomaly scores for sensor readings and takes automated recovery actions if scores cross a threshold. Having this type of approach where they detect an anomaly and automatically respond to it directly reduces the amount of downtime, showing how machine learning fault detection can be integrated within a system.

DeMedeiros et al. (2023) present an in-depth survey of AI-based anomaly detection methods for IoT and sensor networks using supervised, unsupervised, and semi-supervised approaches. The survey finds that unsupervised methods are most useful in remote autonomous systems where it is often impractical to collect large amounts of fault data. This is appropriate for a drone monitoring scenario where normal flight telemetry is easily available but specifically labelled faults are harder to find. The authors point out that the definition of an anomaly varies greatly between studies which makes it difficult to compare results directly. Some studies treat anomalies as discrete attack events, some as gradual sensor drift, some as any deviation from a statistical baseline. That definitional inconsistency is something to keep in mind when interpreting accuracy figures across other papers.

Furthermore, Rafique et al. (2024) explore machine learning and deep learning methods for IoT network anomaly detection, finding that Random Forest and deep neural networks consistently outperform simpler models, achieving detection accuracy above 99% under controlled test conditions. But they also mention that real-world deployment usually reduces this accuracy significantly. The sensor data in a live system contains noise, missing values, and distribution shifts that are absent in training data. This is an important limitation for applying

these techniques to real drone telemetry. They also note in their review that most published results are based on a small number of benchmark datasets, and that models that seem strong in the literature may not be able to reproduce the same accuracy with other data sets. A reasonable rule of thumb when planning a real deployment is to treat published figures as a threshold, and not a guarantee.

Collectively these studies show the potential of machine learning for anomaly detection, but also a common limitation. Most approaches are evaluated in controlled settings on historical datasets, rather than on live systems generating telemetry in real-time. Evaluating on static datasets is great because it makes it easy to compare results, but it hides a lot of the practical complexity that real systems have to deal with constantly: sensor dropout, varying sample rates, mode changes during flight, etc.

Comparing the three studies in this theme directly, Devi et al.'s Isolation Forest approach is unsupervised and lightweight enough to plausibly run in real time, but is only demonstrated on a static IoT dataset. DeMedeiros et al.'s survey explains why this matters: unsupervised methods are favoured in the literature specifically because labelled fault data is hard to obtain, which is exactly the situation a drone monitoring pipeline is in. Rafique et al.'s supervised deep learning results are the most accurate on paper, but also the most fragile outside the lab, since supervised models depend on labelled training data that will not resemble live, noisy drone telemetry. The practical implication for this thesis is that unsupervised or rule-based approaches are more realistic starting points than supervised deep learning, given the absence of labelled fault data from a physical Tello drone.

While anomaly detection with machine learning has been demonstrated to work well in controlled conditions, there is little research on applying these approaches to live telemetry from physical autonomous systems. The majority of studies test algorithms on static datasets, rather than real-time data streams, and therefore have limited relevance to systems such as drones that produce continuous and dynamic telemetry in differing conditions. The integration of anomaly detection into larger real-time monitoring frameworks is also underexplored. More work needs to be done to not only utilise ML techniques such as the isolation forest on static data, but as part of a real autonomous system streaming real-time data.

2.5 Threshold-Based and Adaptive Fault Detection

Machine learning dominates the recent literature on anomaly detection, as the theme above shows. But RQ2 in this thesis is built as threshold-based Prometheus alerting rules run against

raw telemetry, not a trained model. This is a simpler approach than almost anything reviewed so far, and none of the studies above really speak to it. This theme looks at work specific to threshold- and rule-based detection, and at how those thresholds should be set and calibrated on a platform that moves.

The problem with fixed thresholds already came up earlier in this review with Yousuf et al. (2024): a threshold tuned for a stable factory floor does not survive contact with a platform where ‘normal’ shifts with flight mode, payload and weather. Preliminary testing against a physical DJI Tello demonstrated this directly. A placeholder threshold of 60 °C on the onboard temperature metric fired constantly, because the Tello’s board sensor reads above that value while idling on a desk. The alerting worked end to end. The threshold, which had no baseline behind it, was wrong. A number that has not been checked against real data fails as soon as it meets some.

Smallman & Cook (2013) make a related point from the human-factors side. They argue that purely reactive, threshold-based displays do not scale to complex supervisory work, and that trend-based interfaces, ones that show the rate and direction a value is moving rather than only whether it has crossed a line, lead to better decisions in both industrial process control and military command-and-control. Their work is about display design, not automated alerting, but it is the same argument already made in this review about PromQL: a threshold shows whether something is wrong now, while a trend shows whether it is about to be. The overheating false positive from RQ2 testing makes the point well. A single reading of 92 °C looks bad on its own, but the trend of that reading across the session would show whether it was climbing toward failure or sitting at a high, stable baseline.

Ghafouri et al. (2016) put a formal shape on the trade-off this theme keeps circling. Using a game-theoretic attacker-defender model, they show that a fixed detection threshold in a changing system forces a choice between how fast you detect a fault and how many false alarms you get, and that no fixed threshold can be good at both. Their main contribution is an algorithm for computing thresholds that change over time instead, what they call adaptive thresholds, and showing these beat any fixed one. Their case study is a water-distribution network, not a UAV, but the core problem is the same one behind the alerting rule used in RQ2 testing: a detector deciding whether a reading is a fault or just normal variation, using a threshold that never adapts. Their result explains, in general terms, exactly why that 60 °C threshold misfired. A fixed threshold picked without reference to how the system actually behaves cannot minimise detection delay and false alarms at the same time. That’s exactly what happened.

Mazare et al. (2024) apply an adaptive threshold directly to a quadrotor. They pair an

Extended Kalman Filter fault detector with a fixed-time sliding-mode controller, and rather than needing the system's uncertainty bounds set in advance, which is really all an assumed threshold is, their method estimates that bound on the fly and adjusts the threshold to match, while also commanding a recovery response. This is the closest thing in the literature to what RQ2 is trying to do: detect a fault on a physical quadrotor with a threshold that adapts to how the platform actually behaves, instead of a number chosen in advance. However, their threshold comes from an analytical dynamic model of the UAV, which assumes a reasonably accurate model of the platform is available. A low-cost consumer drone such as the DJI Tello has no such model, which is why this thesis derives its thresholds empirically from baseline flight data instead.

Wang et al. (2025) tackle a related problem from another angle: an actuator fault detector that keeps working across different flight scenarios, using domain-adversarial neural networks paired with a mixture-of-experts setup so it can adjust as the data distribution shifts between tasks. Tested on real flight experiments rather than just simulation, it beat conventional deep learning baselines because it could adapt to that shift instead of assuming one fixed idea of 'normal.' That backs up a point made earlier in this review, that a drone's normal operating range moves with flight mode, and shows there is a growing body of UAV-specific work on this problem, even if none of it yet uses threshold-based alerting the way Prometheus does.

Reis & Reis (2025) built a real-time UAV fault detection pipeline meant to run entirely on cheap edge hardware, a Raspberry Pi 4 or an NVIDIA Jetson Nano. It rebuilds noisy or dropped telemetry using B-spline interpolation before feeding it to a hybrid LSTM-autoencoder and Isolation Forest detector. Tested against real PX4 flight logs with faults injected, it hit 93.6% detection accuracy with end-to-end latency under 50 milliseconds. This is a useful benchmark for RQ2's own latency criterion. If sub-second, onboard detection is achievable on hardware far weaker than a laptop running Prometheus, then detection latency in a Prometheus pipeline is more likely to be limited by its scrape and alert configuration than by the computation itself.

Put together, these four studies point the same way: adaptive thresholds are becoming the norm for fault detection on dynamic and UAV platforms, and the reason is the same detection-delay-versus-false-positive trade-off that tripped up the 60 °C threshold in RQ2 testing. But none of them evaluate a rule-based, Prometheus-style alerting setup like this thesis does. Ghafouri et al.'s thresholds are worked out analytically rather than pulled from real baseline data. Mazare et al. need a characterised dynamic model of the platform. Wang et al. and Reis et al. both rely on trained models rather than thresholds you can actually read and explain. That leaves a specific gap: deriving Prometheus alert thresholds from a physical, uncharacterised consumer drone's own baseline telemetry, which is what the Phase 1 baseline-flight work in the methodology chapter sets out to do.

2.6 Self-Healing in Autonomous Systems

A more advanced approach for ensuring the reliability of remote autonomous systems is the use of self-healing systems which enable automatic recovery from detected faults Ogunmolu et al. (2025). Self-healing systems detect a fault and then respond without sending an alert to an operator and waiting for human intervention. They reconfigure themselves to keep working. This capability is especially important in remote autonomous systems where human intervention may be delayed or impossible. The compromise is that an automatic recovery is only as good as the logic behind it. Sometimes a poorly chosen response can do more damage than the original fault. This is why self-healing research focuses not just on whether to act, but on how to ensure the action taken is the right one.

Devi et al. (2024) demonstrate the use of anomaly detection output to trigger automatic recovery in IoT sensor networks. Once there has been a detected fault through Isolation Forest, the system dynamically reconfigures the network by isolating the faulty component and redistributing its load to healthy nodes, thus allowing the network to continue running as normal. This closed-loop process of detection, response, and recovery significantly reduces downtime and highlights the importance of integrating fault detection with recovery in an automated system. The recovery actions in their system are fairly simple, basically isolating a node and redirecting traffic, however this would have to be modified for a system like a drone. With a failed motor or low battery there is nothing to reroute to. In that case, recovery is less about reconfiguring a network and more about behaviour change such as returning home, slowing down and prioritising landing safely.

Ben Hafaiedh & Ben Slimane (2022) provide a more formal approach to self-healing in autonomous systems, where the fault detection, diagnosis and recovery are defined as three different but connected phases. Their approach uses verification to ensure that the steps to recover the system are safe and do not cause new faults which is an important factor in systems where recovery mistakes can be worse than the original fault. Their model is highly relevant to physical autonomous systems such as drones, where actions of recovery have real-world consequences. The framework also makes it easier to reason about each stage in isolation by separating diagnosis from recovery. The system can check that it will always detect a certain class of fault and, separately, that it will always react with a suitable action. This is more difficult to achieve in conventional self-healing systems where detection and response are combined.

More recently, Alauthman & Al-Hyari (2025) propose a fault detection and self-healing

system for wireless sensor networks that uses machine learning in conjunction with the Flying Fox optimisation algorithm to automatically choose the most suitable recovery strategy based on the current system state. This versatile approach is a significant improvement over previous self-healing systems which typically adopted a fixed recovery strategy regardless of how severe the fault is. The ability to adapt the recovery to the systems real time state is especially important for remote autonomous systems operating in dynamic environments. Their reported results, such as 94.6% fault detection accuracy, around 120 milliseconds average recovery time, and 98.5% network resilience, demonstrate the benefit of combining proactive detection with adaptive recovery. These results are from a wireless sensor network setup rather than a mobile platform, so they would need to be re-validated in a drone context, however the fundamentals of this can be applied in the drone context. While faults can vary in severity, not all faults are the same, and different approaches are needed.

Ogunmolu et al. (2025) propose a hybrid AI framework for fault detection and self-healing in smart manufacturing, combining spiking neural networks, symbolic reasoning and Isolation Forest, validated through hardware-in-loop simulation, fault injection testing and field deployment in automotive and electronics manufacturing. Their results are strong: 97.3% fault detection accuracy and an 89.4% self-healing recovery rate, reducing mean-time-to-repair by 31.7% and improving overall equipment effectiveness by 6.7 points. The limitation they report themselves is telling: material regeneration during recovery took approximately 83 seconds on average, which exceeded the takt time of some high-speed production lines, meaning the recovery was accurate but not always fast enough to matter operationally. This is a useful caution for this thesis's own evaluation criteria: a fault detection and recovery system is not judged on accuracy alone, but on whether detection and recovery both complete within a timeframe that is actually useful for the system being monitored, whether that is a production line or a drone losing altitude.

Despite these advances, most self-healing research focuses on networked or distributed software systems rather than physical autonomous systems operating in real-world environments. The literature also does not delve into the combination of fault detection, recovery and continuous health monitoring in a single integrated system. In most studies, recovery is seen as a separate event triggered by a fault, rather than as one part of a continuous monitoring loop.

Across the four self-healing studies reviewed, there is a clear progression in sophistication: Devi et al.'s network-reconfiguration approach is simple but has no direct analogue for a drone, since there is no healthy neighbour to reroute a failed motor's load to. Ben Hafaiedh and Ben Slimane's formally verified phase separation is the most rigorous but the most abstract, offering no concrete recovery actions. Alauthman and Al-Hyari's adaptive strategy selection

and Ogunmolu et al.'s hybrid AI framework both move toward context-sensitive recovery, but neither is evaluated on a physical mobile platform, and Ogunmolu et al.'s own reported recovery latency shows that even a well-validated recovery mechanism can be too slow for a time-critical system. None of the four evaluate whether the recovery itself succeeded beyond the immediate test condition, which remains an open question this thesis does not attempt to answer.

While self-healing systems have shown great recovery capabilities, there is very little work done by other research professionals on integrating self-healing mechanisms with continuous real-time monitoring in physical autonomous systems. Most of the existing work focuses on the detection or the recovery phase in isolation, with little focus on how both can be embedded within a larger monitoring framework. There is also little work on how to monitor recovery itself, i.e., how a system checks that recovery has indeed succeeded rather than assuming it has.

2.7 Monitoring Frameworks and Observability

Before any of the techniques discussed in this review can work, there needs to be infrastructure collecting and storing the data in the first place. Tools like Prometheus collect system metrics continuously, store them in time-series databases, and make them queryable in real time. Observability is a related but slightly broader idea. Where monitoring tells you whether known things are happening, such as whether the battery is below 20%, observability lets you ask questions such as why a particular flight behaved differently from the last one. For a drone monitoring system, both of these tools are essential.

Boncea & Bacivarov (2016) were the first to officially propose a monitoring architecture specifically designed for IoT reliability. They used time-series databases and tools such as Prometheus, InfluxDB and Graphite to continuously collect and analyse the system's metrics. Their design separates metrics collection, storage, and querying into different layers, allowing for easier scaling and adaptation with the addition of new devices. Prometheus was used as the main collection tool throughout their research. Its pull-based scraping model handles intermittent connectivity by recording missed scrapes rather than silently losing data, where the monitoring server requests metrics from devices at regular intervals. This feature is useful for this research with drones, where connectivity can be lost and regained multiple times during a single flight. A push system would lose data during a dropout, while Prometheus just registers a gap and resumes when the link has been restored.

Prometheus' query language, PromQL, makes it easier to create alert rules and extract useful insights from raw telemetry data, instead of only collecting basic metrics. PromQL allows users to calculate rates of change, moving averages and anomaly thresholds inside the monitoring tool instead of just collecting values. This is very useful for autonomous systems, where trends and rates of change are often more useful than raw sensor readings alone. A battery at 60% is not interesting, but a battery dropping at 3x the expected rate definitely is. Having that kind of insight is exactly what PromQL is built for.

Since this paper was released, Prometheus has become one of the most widely used monitoring tools in both cloud and IoT environments. Barrett et al. (2023) review Prometheus and Grafana as an observability stack in cloud settings, stating that using both of them together offers thorough real-time monitoring with very little configuration overhead. The study emphasises the fact that the dashboard capabilities of Grafana transform raw Prometheus metrics into visual system health views that are easy to interpret and understand. This is an important practical consideration when monitoring a system such as a drone that produces many metrics at the same time. Monitoring of this data would need to be easy and quick in the case of an emergency, unless of course a ML model is handling it instead. The paper also discusses Prometheus and Grafana as being well suited to environments where lots of different data sources are constantly changing. While a drone system is quite different from a containerised cluster, the same idea of cleanly managing and monitoring multiple short-lived data streams is still useful when tracking several drones or multiple flight sessions over time.

More recently, Rafiq et al. (2025) demonstrate an AI and IoT-driven monitoring system integrating Prometheus and Grafana with machine learning models for predictive anomaly detection. Their system uses Prometheus' ability to monitor and then pairs it with advanced analysis to predict potential failures before they occur. They go beyond the normal threshold-based alerts by using machine learning models. This integration of monitoring framework with machine learning is the direction modern system health management is heading and is especially useful for remote autonomous systems such as drones. According to their results, implementing the system across five client networks reduced downtime by around 95% and improved incident resolution times by roughly 90% compared to systems without predictive monitoring. These results come from a managed services environment rather than an autonomous drone system, so they should not be treated as direct performance benchmarks. However, the overall monitoring architecture and approach are still highly applicable to drone-based scenarios.

Read chronologically, Boncea and Bacivarov establish the architectural case for Prometheus-style monitoring, Barrett et al. demonstrate that the same stack works with low configuration overhead in a practical cloud setting, and Rafiq et al. show the next step of pairing that stack

with machine learning for predictive alerts. What none of the three demonstrate is the same stack operating against a telemetry source that itself moves and drops in and out of range, which is the specific condition this thesis tests.

While these tools are widely used in cloud and industrial environments, their use in remote autonomous systems, and especially mobile platforms such as drones, is still very limited. Most of the existing implementations assume stable network connectivity and relatively predictable workloads, both of which cannot be guaranteed during drone operations. There is also relatively little discussion of how to handle telemetry from a platform that moves, changes state, and drops offline occasionally, making the simple task of scraping metrics much more difficult.

Monitoring frameworks such as Prometheus and Grafana are well established in cloud and distributed systems, however, there is very limited research on applying them to remote autonomous systems, especially physical, mobile platforms such as drones. There is a lack of practical implementations to demonstrate how these tools can maintain continuous health visibility in environments with variable connectivity and changing operating conditions, which is exactly the type of environment a drone operates in.

A further gap in this theme is evaluation methodology. Much of the literature above reports that a monitoring or alerting system “works,” without stating how detection latency or false-positive rate were measured, or over what evaluation period. Prometheus’ own operational guidance treats alerting as inherently imperfect: the goal is to catch symptoms that matter while accepting that perfect, instantaneous detection is not possible. Typical alert evaluation latency in production deployments is reported as roughly 30 seconds to 2 minutes, tunable per rule. This framing is directly relevant to RQ2’s evaluation criteria (detection latency and false-positive rate), but is largely absent from the academic literature reviewed above, which tends to report a single accuracy figure rather than latency/false-positive trade-offs measured against a live system.

2.8 Research Gap

Existing research addresses reliability, fault detection, anomaly detection, self-healing and monitoring frameworks as separate areas, but there is a shortage of literature that integrates all these in the context of real-time monitoring for physical remote autonomous systems. While each individual area has advanced, the integration of continuous health monitoring,

fault detection with the use of ML and automated recovery into a remote autonomous system remains unexplored. In particular, no practical implementations exist using tools such as Prometheus on systems that operate in dynamic real-world environments such as drone flight. Most existing studies evaluate their approaches in controlled or static environments. This limits how well the findings apply to autonomous platforms that operate in changing conditions with temporary connectivity loss, and ones that make real-time health decisions.

The aim of this research is to investigate the use of monitoring frameworks, like Prometheus, to assess and ensure the health and reliability of remote autonomous systems. A DJI Tello drone represents a remote autonomous system, and its real-time telemetry is fed into Prometheus. The aim is to evaluate whether this approach is effective for fault detection and continuous health visibility in a physical, mobile autonomous system. This is the gap identified across the themes in this review.

More specifically, RQ2 targets the threshold-calibration gap identified above: rather than relying on pre-labelled data or an offline-trained ML model, this research derives fault-detection thresholds empirically from a physical drone's own baseline telemetry, and evaluates a rule-based Prometheus alerting configuration against live data, including detection latency and false-positive rate, addressed nowhere in the literature reviewed for a mobile robotic platform. Preliminary testing produced one concrete, evidenced instance of this exact problem (an assumed 60 °C temperature threshold producing a sustained false positive against real Tello telemetry), which motivates the Phase 1 baseline-calibration approach set out in the methodology.

3 Methodology

3.1 Overview

This section sets out the research methodology: the aims and questions the work addresses, the method used to answer each research question, the schedule and milestones against which progress is tracked, the ethical obligations that apply, and the threats to the validity of the results.

The research is conducted as **two sequential experimental studies** that share one design and one artefact.

- **Study 1 (RQ1)** validated the monitoring pipeline architecture using replayed telemetry from a publicly available, pre-labelled drone dataset. Because ground-truth health state was supplied by the dataset, the only thing under test was whether the pipeline could collect, store and surface health state in real time.
- **Study 2 (RQ2)** removes the labels. The same pipeline is connected to a physical DJI Tello drone, fault thresholds are derived empirically from the drone's own baseline telemetry, and Prometheus alerting rules must determine health state autonomously from raw sensor readings.

There is a reason for running the studies in this order, and it is not only that the project spans two trimesters. Study 1 eliminates the pipeline architecture as a source of error, so any detection failure in Study 2 can be attributed to the detection logic, meaning the thresholds and alerting rules, rather than to the ingestion, storage or query layers beneath them. Establishing the instrument before testing the hypothesis is standard practice in experimental software engineering (Wohlin et al. 2012), and it is what allows Study 2 to isolate threshold calibration as the variable of interest.

3.1.1 Candidate Methods and Selection

Five methods were considered. Table 1 compares them and states why each was or was not adopted.

Table 1: Comparison of Candidate Research Methods

Method	Strengths	Weaknesses	Decision and reason
Experimental Research (Wohlin et al. 2012)	Produces real, measurable evidence; repeatable under controlled conditions	May not generalise beyond the test environment; needs controllable fault conditions	Selected (primary). The research questions ask for latency and false positive figures, which only controlled measurement on a running system can produce (Yousuf et al. 2024, Paradeshi et al. 2022).
Design Science Research (Hevner et al. 2004)	Produces a reusable artefact; iterative build and evaluate cycle	Evaluation can be subjective unless criteria are fixed in advance	Selected (secondary). The monitoring pipeline is a contribution in itself, not only an instrument, so it needs a method that evaluates artefacts (Boncea & Bacivarov 2016).
Simulation / Modelling	Safe and repeatable; no hardware cost	Does not reproduce real hardware behaviour, Wi-Fi variability, thermal drift or battery ageing	Rejected. The gap concerns the absence of evidence from physical mobile platforms. Simulating the platform would reproduce the gap instead of closing it.
Case Study / Field Observation	High realism; captures operational context	No controlled fault injection; poor repeatability	Rejected. RQ2.3 needs known fault onset times, which uncontrolled observation cannot supply.
Literature Review	Establishes grounding; identifies gaps and prior approaches	Produces no original data; cannot validate an artefact	Supporting input only. Used to establish the gap and select tooling and metrics.

Experimental research (primary). The primary method is a controlled, repeated-measures fault-injection experiment. Baseline flights under normal conditions are the control condition and flights with deliberately induced degradation are the treatment conditions. The independent variable is the class of induced fault. The dependent variables are detection outcome, detection latency, and alerts fired when no fault was induced. A latency figure needs a known fault onset time. A false positive rate needs a period of flight certified free of faults. Both are properties of a controlled experiment and neither survives in uncontrolled observation.

Design science research (secondary). The monitoring pipeline is itself a contribution, and DSR provides an established structure for building and evaluating such artefacts (Hevner et al. 2004). Boncea & Bacivarov (2016) used the same approach to propose the Prometheus-based architecture this thesis builds on. The artefact is the pipeline: telemetry source → Python exporter → Prometheus → alerting rules → Grafana dashboard. It was developed across two

DSR iterations, one per study.

3.1.2 Experimental Platform and Tooling

Table 2: Required Resources

Resource	Purpose	Access / version
DJI Tello drone	Representative RAS for live telemetry collection (Study 2)	Provided by Deakin University
Laptop / PC	Runs exporter, Prometheus, Grafana and session logger on one host	Personally owned; Windows 11
Wi-Fi (2.4 GHz)	Tello SDK communication over UDP	Direct Tello access point
Prometheus	Time-series collection, storage and alert rule evaluation	Open-source
Grafana	Dashboard visualisation of telemetry and alert state	Open-source
Python + <code>djitellopy</code>	Telemetry extraction and Prometheus exporter	Open-source; Python 3.11.9
Drone Telemetry Tampering Dataset v2	Pre-labelled telemetry for Study 1	Public, via Kaggle (Ekanayakadevika 2024); CC BY-SA 4.0

Prometheus was selected for its pull-based scrape model, native time-series storage and PromQL query language, which supports the threshold-based alerting RQ2 requires. It is also the framework most widely reported in comparable monitoring architectures (Boncea & Bacivarov 2016), which makes these results directly comparable to that prior work. Grafana was selected as the visualisation layer because it integrates natively with Prometheus and is likewise established in the reviewed literature (Ben Hafaiedh & Ben Slimane 2022). The exporter is written in Python for its suitability to telemetry handling and its access to `djitellopy`.

A push-based stack (Telegraf/InfluxDB) would avoid the scrape-interval latency floor discussed below, and log-oriented tooling (ELK) suits event data better than numeric time series. Both were rejected, the former for being less widely evidenced in the RAS monitoring literature and the latter for being poorly matched to continuously sampled numeric telemetry. Managed cloud IoT platforms were rejected on principle, since the gap this thesis addresses is the absence of monitoring approaches that work *without* cloud dependency.

One trade-off is worth stating early. Prometheus’ pull model imposes a hard lower bound on detection latency equal to the scrape interval, however quickly the exporter updates. Study 1 showed this directly: metric values updated at the exporter every second, but observed detection latency was governed by the 15-second scrape cycle. Study 2 therefore uses a 1 s scrape interval,

accepting the higher storage and CPU cost in exchange for a latency floor compatible with the targets in Section 3.5.

3.2 Aims

The research has four aims, which together define what the two studies are intended to produce.

1. **Design a monitoring pipeline for remote autonomous systems** that collects, stores and surfaces health telemetry in real time using open-source components and no cloud dependency.
2. **Establish which telemetry metrics represent the operational health** of a remote autonomous system, and demonstrate that the pipeline can track transitions between healthy and faulted states.
3. **Derive fault detection thresholds empirically** from a platform's own measured baseline behaviour, rather than assuming values carried over from other domains, and state the derivation procedure so that it transfers to other platforms.
4. **Evaluate the resulting detection performance** on physical hardware under controlled fault conditions, reporting detection rate, detection latency and false positive rate against targets fixed in advance.

Aims 1 and 2 are addressed by Study 1 and answer RQ1. Aims 3 and 4 are addressed by Study 2 and answer RQ2.

3.3 Research Questions

Two research questions guide the study, each decomposed into measurable sub-questions.

RQ1: *How can monitoring frameworks be used to assess and maintain the health of remote autonomous systems?*

- **RQ1.1:** Which telemetry metrics must a monitoring pipeline expose in order to represent the operational health state of a remote autonomous system?
- **RQ1.2:** Can a pull-based time-series monitoring framework track transitions between healthy and faulted states in RAS telemetry in real time?

RQ2: *How can system metrics such as battery level, connectivity and performance be used to detect faults in remote autonomous systems?*

- **RQ2.1:** What metric thresholds reliably distinguish normal operation from fault conditions in a physical drone system, and how should those thresholds be derived?
- **RQ2.2:** How accurately can Prometheus alerting rules detect fault conditions from raw telemetry without pre-labelled data?
- **RQ2.3:** What are the detection latency and false positive rate of the threshold-based alerting system under controlled fault scenarios?

These questions target the gap established in Section 2.8. The literature reviewed rarely integrates real-time telemetry pipelines with physical autonomous systems operating outside controlled industrial settings (Boncea & Bacivarov 2016, Devi et al. 2024), and where threshold-based detection is used, thresholds are typically assumed rather than measured from the platform under observation (Yousuf et al. 2024). RQ1 addresses the integration half of that gap. RQ2 addresses the calibration half, which is the more consequential of the two. Preliminary testing produced a concrete instance of why: an assumed 60 °C temperature threshold, carried over from industrial motor monitoring conventions, produced a sustained false positive against real Tello telemetry, because the Tello’s normal operating range differs from the assumption. That observation is the empirical motivation for the baseline-calibration procedure in Section 3.5.

Table 3 maps each sub-question to the method, data, instrument, analysis and success criterion that address it.

3.4 RQ1 Methodology

Study 1 tests whether the pipeline can *observe* health state. It does not test whether the pipeline can *determine* health state, which is RQ2’s concern.

Table 3: Research Design Matrix

Sub-question	Data source	Instrument	Analysis	Success criterion
RQ1.1	Drone Telemetry Tampering Dataset v2	Python exporter metric schema	Mapping of exposed metrics to RAS health dimensions	All identified health dimensions represented by at least one exposed metric
RQ1.2	Dataset v2 (six labelled anomaly phases)	Exporter → Prometheus → Grafana	PromQL transition counting against dataset labels	100% of labelled fault-phase transitions reflected in <code>ras_health_status</code>
RQ2.1	5 Tello baseline flight sessions	Live Tello SDK exporter at 1 s scrape	Per-metric distribution analysis; percentile and operational-limit derivation	Threshold defined for every monitored metric, each traceable to baseline data
RQ2.2	9 Tello fault-injection runs (3 types × 3 reps)	Prometheus alerting rules (<code>rules.yml</code>)	Confusion matrix; detection rate and precision	≥8 of 9 induced faults detected by the correct rule
RQ2.3	Same 9 runs plus 5 baseline sessions	Alert firing times vs. raw telemetry event log	Latency distribution (median, IQR, max); false alerts per unit flight time	Median latency ≤5 s, max ≤10 s; ≤1 false alert per 10 min normal flight

Data source. Telemetry came from the Drone Telemetry Tampering Dataset v2 (Ekanayakadevika 2024), a publicly available corpus of drone flight telemetry containing six labelled anomaly phases interleaved with normal operation. A pre-labelled dataset was chosen deliberately: because each row carries a ground-truth health state, any discrepancy between the dataset labels and what the pipeline surfaced is attributable to the pipeline and nothing else.

Instrument. A Python exporter reads the dataset row by row and republishes each record as Prometheus metrics on an HTTP endpoint, replaying the recorded flight as though it were arriving live. Alongside the raw telemetry the exporter publishes `ras_health_status`, a composite state metric taken directly from the dataset label. Prometheus scrapes the endpoint on a fixed interval and Grafana renders the metrics and the health state.

Protocol. The full dataset is replayed end to end in a single continuous run at a 15 s scrape interval. Metric coverage is checked throughout with `up` and `per-metric absent()` queries. Every labelled phase transition in the source data is then compared against the corresponding change in `ras_health_status` as recorded in Prometheus.

Analysis. RQ1.1 is answered by mapping each exposed metric to the health dimensions identified in the literature review, and checking that every dimension is represented. RQ1.2 is answered by counting labelled phase transitions in the dataset and the corresponding transitions recorded in Prometheus; a transition is counted as tracked only if it appears in `ras_health_status` within one scrape interval of the labelled change.

Table 4: Study 1 Evaluation Criteria (RQ1)

Criterion	Operational definition	Target	Measured by
Metric coverage	All defined health metrics present and non-stale in Prometheus for the full replay	100% of metrics	up, absent ()
Transition fidelity	Labelled fault-phase transitions reflected by a corresponding change in <code>ras.health.status</code>	100% of transitions	PromQL transition count vs. dataset labels
Detection latency	Delay between a metric value changing at the exporter and that change being queryable in Prometheus	≤ 1 scrape interval	Exporter log vs. Prometheus sample timestamps
Dashboard completeness	All monitored metrics and the health-state panel visible on one screen without scrolling	All metrics plotted	Dashboard inspection

Limitation carried into Study 2. Because health state was read from a dataset label rather than computed from raw values, Study 1 says nothing about whether the system can recognise a fault it has not been told about. That limitation is what Study 2 exists to address, and it is the reason the two studies are sequential.

3.5 RQ2 Methodology

Study 2 removes the labels. The same pipeline is connected to a physical DJI Tello, and health state must be computed by Prometheus alerting rules from raw telemetry alone.

3.5.1 Platform Telemetry Constraints

The Tello’s available telemetry differs materially from that of the GPS-equipped platforms described in much of the reviewed literature, and from the fields in the Study 1 dataset. The standard Tello carries **no GNSS receiver**, so geospatial metrics are unavailable, and Wi-Fi link quality is exposed as a signal-to-noise ratio query rather than a continuously sampled RSSI value. Table 5 lists the fields the SDK does expose. This constraint shapes the metric set and returns as a threat to external validity in Section 3.8.

Table 5: Tello SDK Telemetry Fields Used in Study 2

Field	Accessor	Unit	Role
Battery level	<code>get_battery()</code>	%	Primary depletion fault signal
Flight time	<code>get_flight_time()</code>	s	Session duration; depletion rate denominator
Height	<code>get_height()</code>	cm	Barometrically derived height; drift indicator rather than measured altitude
ToF distance	<code>get_distance_tof()</code>	cm	Primary altitude-hold signal
Barometer	<code>get_barometer()</code>	cm	Altitude drift detection
Temperature (high / low)	<code>get_highest_temperature()</code> , <code>get_lowest_temperature()</code>	°C	Thermal state; settles toward an operating point in flight
Acceleration x,y,z	<code>get_acceleration_x/y/z()</code>	0.001g	Derived instability metric
Attitude (pitch, roll, yaw)	<code>get_pitch()</code> , <code>get_roll()</code> , <code>get_yaw()</code>	deg	Derived instability metric
Velocity x,y,z	<code>get_speed_x/y/z()</code>	dm/s	Recorded for completeness; near zero at 1 Hz even during commanded translation
Wi-Fi SNR	<code>query_wifi_signal_noise_ratio()</code>	dB	Link degradation fault signal

3.5.2 Data Collection Protocol

All sessions use a 1 s Prometheus scrape interval. The exporter, Prometheus and session logger run on a single host, so every timestamp comes from one system clock and clock synchronisation between devices cannot introduce measurement error.

Phase 1: Baseline (RQ2.1). Five flight sessions under normal operating conditions, each of four minutes. Flights follow a fixed manoeuvre script (take-off, hover, translation, rotation, hover, land) so sessions are comparable. No faults are induced and no interventions are made. The purpose is to establish the empirical distribution of every monitored metric under normal operation, which is what the thresholds are derived from.

Three controls hold the sessions comparable to one another. Each session begins on a fully charged battery, because percentage-per-minute depletion is not linear across the discharge curve and a part-charged start samples a different region of it. Batteries are alternated between sessions so that battery identity is not confounded with session order. The aircraft is returned to ambient temperature before each session, since residual heat from a preceding flight displaces the thermal distribution of the next one.

Four-minute sessions sample the upper region of the discharge curve only. The lower region, approaching the automatic low-battery landing, is characterised by the Phase 2 depletion runs, which continue until that failsafe engages.

Phase 2: Fault injection (RQ2.2, RQ2.3). Nine runs, made up of three fault classes with three repetitions each.

1. **Battery depletion.** Flight continues until the Tello's automatic low-battery landing engages. Tests whether the depletion rule fires with enough lead time before the failsafe.
2. **Link degradation.** The wireless interface on the ground station is disabled while the aircraft is airborne, severing the control link at a known instant. Progressive separation and obstruction of line of sight were trialled first, but the transmit range of the aircraft exceeded the available indoor test area, so the link could not be degraded far enough to cross any threshold. Tests the liveness rules.
3. **Thermal stress.** The aircraft is held powered on the ground for approximately eight minutes, without propellers turning, and then launched into the standard manoeuvre script. Baseline measurement established that onboard temperature settles toward an operating point during flight rather than rising under load, because the propeller wash cools the

airframe. Manoeuvring alone therefore cannot produce a thermal fault, whereas a powered ground soak raises the starting temperature above the range the baseline flights covered. Tests the temperature rules.

Ground-truth marking. Live telemetry carries no labels, so fault onset has to be established independently of the alerting system being measured. Two timestamps are recorded per run: t_{induce} , the moment the operator begins inducing the fault, written to the session log as an event marker by the control script; and t_{onset} , the first crossing of the defined threshold in the raw 1 Hz telemetry log that persists for at least the hold duration applied by the rule, worked out retrospectively during analysis.

The hold duration is part of the definition because a shorter crossing cannot raise an alert, and anchoring latency to one would measure the interval between two unrelated events. Crossings below that duration are recorded separately as transient crossings, since their frequency indicates how much of the false positive suppression is attributable to the hold rather than to the threshold value.

Detection latency is measured from t_{onset} . Measuring from t_{induce} would fold the drone's physical response time and the operator's reaction into the figure, and neither of those is part of what RQ2.3 asks about. t_{induce} is still recorded, as context and to confirm each run's fault was induced as intended.

Data recorded per session. Each session is logged under four headings. The first three make a session identifiable and comparable to the others; the fourth is the measurement itself.

- **Session identity.** A unique session identifier, and the date and time of the flight.
- **Conditions.** The venue, whether the flight was indoors or outdoors, and the ambient temperature.
- **Hardware state.** The Tello firmware version, the battery identifier, and that battery's approximate cycle count.
- **Measurements.** The full 1 Hz raw telemetry log, every Prometheus alert firing and resolution timestamp, the operator event markers, and the Prometheus system logs.

Conditions and hardware state are recorded because they vary between sessions and affect the readings. Ambient temperature shifts the thermal baseline, and battery age changes the depletion rate. Logging both means an outlier session can be explained rather than discarded.

3.5.3 Threshold Derivation Procedure

Study 1's main limitation was that health state came from dataset labels instead of being computed. Removing that dependency makes threshold derivation the central methodological problem of this thesis, not an implementation detail.

A single uniform rule will not work, because the monitored metrics do not behave the same way as one another. Battery level, for example, falls steadily throughout every flight by design, so a percentile of its distribution would say nothing useful about when the drone is actually at risk. Table 6 groups metrics by how they behave and gives each group its own derivation rule and rationale.

Every rule also uses a Prometheus `for` condition, requiring the threshold to remain active for 5 seconds before an alert is triggered. This reduces false alarms caused by short-term fluctuations, which are common in telemetry sampled from a moving platform, but adds 5 seconds to the detection time. This additional delay is considered in the latency targets presented in Table 7.

Time-of-flight distance is measured and reported but no threshold is derived from it. Its upper tail arises from the aircraft passing over lower floor area in one particular room, which is a property of the venue rather than of the platform and would not transfer to another flight space.

The threshold values themselves are reported with the Study 2 results in Section 5, since they are findings rather than design decisions.

3.5.4 Data Analysis Plan

Four measures are computed from the session logs. Each is defined here so that any figure reported in Section 5 can be traced back to the raw data and recalculated.

Detection outcome. Every fault run is classified into one of three outcomes. A *true positive* means the rule matching the induced fault fired. A *false negative* means no rule fired. A *misclassification* means a rule fired, but for the wrong fault class. The nine runs are collected into a confusion matrix, which gives the detection rate and precision.

Table 6: Threshold Derivation Rules by Metric Class

Metric class	Metrics	Derivation rule	Rationale
Monotonic / depleting	Battery level	Operational limit plus safety margin: alert above the firmware auto-land point, with the margin sized from the mean depletion rate observed in Phase 1	The quantity declines by design, so no percentile of its distribution is meaningful. What matters is keeping enough lead time to act before the failsafe.
Stationary / bounded	Wi-Fi SNR	Empirical 1st or 99th percentile of the pooled Phase 1 distribution, per metric direction	Percentiles assume nothing about normality. Telemetry from a mobile platform is skewed and multi-modal across flight modes, so mean $\pm k\sigma$ would put the bound in the wrong place.
Settling	High and low onboard temperature	99th percentile of the settled region, taken as the final 90 s of each Phase 1 session, per metric direction	The quantity converges toward an operating point rather than varying about a mean, so most samples in a short session describe the warm-up transient rather than steady operation. A percentile over the whole session is displaced below the range the aircraft occupies once settled.
Drift / relative	Barometric height, derived height	Deviation from the metric's own trailing 60 s mean, with the threshold at the 99th percentile of the pooled Phase 1 deviation distribution	The absolute reading tracks atmospheric pressure rather than the aircraft, so a pooled percentile band would encode the prevailing weather. Deviation from the metric's own recent mean is comparable across sessions; the raw value is not.
Derived variance	Attitude and acceleration instability	Rolling standard deviation over a 5 s window, with the threshold at the 99th percentile of the Phase 1 rolling-standard-deviation distribution	Instantaneous attitude values mean little on their own during normal manoeuvring. Dispersion is what separates controlled flight from loss of stability.
Liveness	Target reachability, metric staleness	Binary: <code>up == 0</code> or <code>absent()</code> sustained beyond two scrape intervals	Missing data is itself a fault condition, and cannot be expressed as a numeric bound.

Detection latency. The gap between the fault appearing in the telemetry and the alert firing, calculated as $t_{\text{alert}} - t_{\text{onset}}$. Reported as a median, an interquartile range and a maximum across all detected faults.

False positives. An alert that fires while the drone is behaving normally. This can happen in two places: during a baseline session, or during a fault run before the fault has been induced. Both are counted. The result is expressed as a rate, false alerts per 10 min of normal flight, rather than a raw count, so the figure stays comparable if the number of sessions changes.

The rate is measured on normal flights held out from threshold derivation, together with the pre-induction segment of each fault run. Measuring it on the derivation sessions themselves would be circular, since a 99th-percentile threshold is exceeded by approximately one percent of its own derivation data by construction.

Metric coverage. The proportion of monitored metrics that are present and up to date in Prometheus for the full duration of a session, checked with `up` and `per-metric absent()` queries. This confirms the other three measures were computed from complete data.

Only descriptive statistics are used. Five baseline sessions and nine fault runs are too few to support significance testing, so results are reported as medians with ranges, accompanied by the individual per-run figures. This lets the reader judge the variability directly rather than relying on a summary statistic the sample cannot justify. The small sample is a consequence of the experimental design, and its effect on what can be claimed is addressed in Section 3.8.

The measures above define what is calculated. Table 7 states the target each must meet for the study to be judged successful, fixed before any data was collected.

All criteria in both studies were fixed before the corresponding data was collected, and are expressed as numeric, observable targets. This is a requirement of DSR, where artefact evaluation is subjective unless criteria are defined in advance (Hevner et al. 2004).

3.6 Timeline and Milestones

Table 8 states the milestones against which progress was tracked across the two trimesters, together with the evidence recorded at each one.

Table 7: Study 2 Evaluation Criteria (RQ2)

Criterion	Operational definition	Target	Measured by
Threshold traceability	Every alerting rule threshold derived from Phase 1 baseline data by a stated rule, with no assumed values	100% of rules	Threshold table vs. baseline corpus
Detection rate	Induced faults for which the correct alerting rule fired	≥ 8 of 9 runs	Confusion matrix
Detection latency	$t_{\text{alert}} - t_{\text{onset}}$	Median ≤ 5 s; max ≤ 10 s	Alert log vs. raw telemetry log
False positive rate	False alerts per 10 min of normal flight, pooled across baseline sessions	≤ 1 per 10 min	Alert log vs. baseline sessions
Metric coverage	All monitored metrics present and non-stale for the full duration of every session	$\geq 99\%$ of samples	up, absent ()
Dashboard completeness	All monitored metrics, current alert state and fault annotations visible on one screen	All metrics and alert state	Dashboard inspection

Table 8: Research Milestones

#	Milestone	Deliverable / evidence	Target	Status
M1	Pipeline validated on labelled data	Study 1 results, Section 4	T1	Complete
M2	Live Tello telemetry reaching Prometheus	Working exporter; target health page	19 Jul 2026	Complete
M3	Baseline data collection complete	5 archived session logs	Sun 30 Aug 2026	Complete
M4	Thresholds derived and rules encoded	Threshold table; <code>rules.yml</code>	Sun 30 Aug 2026	Complete
M5	Fault injection complete	9 annotated run logs	Sun 7 Sep 2026	Complete
M6	Analysis complete	Confusion matrix; latency and FP results	Fri 11 Sep 2026	Complete
M7	Full draft to supervisor	Complete thesis draft	Mon 14 Sep 2026	Complete
M8	Final submission	Final thesis and presentation	Mon 21 Sep 2026	On track

Table 9: Risks and Mitigations

Risk	L / I	Mitigation	Status
Threshold miscalibration causing false positives	High / High	Thresholds derived from Phase 1 baseline data by the rules in Table 6, never assumed; <code>for</code> : hold durations suppress transients; thresholds pooled across all five baseline sessions rather than fitted to one; derived values validated against normal flights held out from the derivation	Realised in preliminary testing (60 °C assumption) and confirmed against baseline data
Metric assigned to the wrong behavioural class	Med / High	Metric class assignments in Table 6 tested by applying the derived threshold to held-out normal flights, since a misclassification is not visible from the derivation rule alone	Realised for barometric height and for onboard temperature; both reclassified
Tello hardware failure or unavailability	Med / High	Drone sourced from the Deakin equipment pool early in the trimester; spare batteries maintained; fallback to extended dataset-based experimentation preserves RQ2.1 and RQ2.2	Not realised; one airframe served the whole study
Wi-Fi interference degrading telemetry	Med / Med	Flights conducted on the Tello's own access point indoors; multiple sessions per phase; outlier sessions reported rather than silently discarded	Controlled
Scrape interval too coarse for real-time detection	Low / Med	Scrape interval reduced from 15 s to 1 s for Study 2; storage overhead measured and reported	Mitigated
Insufficient fault range achievable indoors	Med / Med	Wi-Fi degradation supplemented by deliberate line-of-sight occlusion; outdoor contingency sessions available under CASA compliance	Realised ; transmit range exceeded the indoor test area, so the link fault was induced at the ground station instead
Small sample limiting statistical claims	High / Low	Descriptive statistics only, with per-run figures reported; limitation stated explicitly	Accepted
Time constraints in a single trimester	Med / High	Two-week buffer between M6 and M7; Study 2 scoped to three fault classes	Mitigated; all milestones met

The contingency in Table 9, under which analysis would have reverted to an extended dataset-based experiment, was not required. Both collection phases completed on the physical aircraft.

3.7 Ethical Considerations

3.7.1 Human Participants and Ethics Determination

This research involves no human participants, no personal data and no observation of people. All data is machine-generated telemetry from a consumer drone, so the project does not constitute human research under the National Statement on Ethical Conduct in Human Research and requires no approval from either a Faculty Human Ethics Advisory Group (low-risk review) or the Deakin University Human Research Ethics Committee (DUHREC).

3.7.2 Flight Safety and Regulatory Compliance

Operating a powered aerial vehicle carries physical risk to people and property, which is managed as follows.

Indoor flights (primary). All flights contributing data to this study were conducted indoors at a private residence. Two exploratory sessions were flown outdoors on 28 August 2026 under the CASA conditions described below; both were excluded from analysis because baseline and fault-injection runs must share a single environment, and both are retained in the repository as rejected sessions. Civil Aviation Safety Authority (CASA) rules governing outdoor airspace do not apply indoors, so for the sessions used here the controlling considerations are venue and bystander safety. Permission to fly is obtained from the space’s custodian before each session. Flights are conducted in a cleared area with no uninvolved persons present, propeller guards fitted at all times, and altitude limited by ceiling clearance. The Tello’s automatic low-battery landing failsafe stays enabled in every session, including battery depletion runs, where it is the object of measurement rather than a constraint to disable.

Outdoor flights (contingency, not invoked). Every session reported in this thesis was flown indoors. Had outdoor sessions been needed to obtain a wider Wi-Fi degradation range, they

would have complied with CASA recreational rules for very small remotely piloted aircraft (the Tello's take-off weight is approximately 80 g): below 120 m AGL, in daylight, within visual line of sight, clear of populous areas and emergency operations, and outside the exclusion radius of controlled aerodromes. Current CASA guidance would have been checked immediately before any such session rather than relied on from this document, as the rules are subject to amendment, and no session would have been flown in rain or in wind exceeding the manufacturer's stated limit.

3.7.3 Data Management and Privacy

No personally identifiable information is collected at any stage. The Tello carries no GNSS receiver, so no location data is recorded, and although the airframe has a camera, the video stream is not enabled, captured or stored in any session. Only numeric telemetry is collected.

Telemetry, session logs and configuration files are stored locally on password-protected, full-disk-encrypted hardware, with backups to a private version-controlled repository accessible only to the researcher and supervisor during the project. Data is retained for the minimum period required by university policy following submission. On submission the telemetry and configuration artefacts are released publicly as described in Section 3.9; because the data contains no personal or sensitive information, no de-identification is required before release.

3.7.4 Research Integrity and Use of Generative AI

All results are reported as observed, including failures, missed detections and false positives. The 60 °C false positive encountered during preliminary testing is reported in full instead of discretely corrected, because it is evidence for the central argument of RQ2. Negative and partial results are reported with the same prominence as successful ones.

Generative AI tools were used in a defined and limited capacity, consistent with Deakin University's requirements on acknowledging AI assistance. Claude (Anthropic) was used to assist in producing the software instruments for both studies: the Study 1 replay exporter, and the Study 2 session harness, session validator and threshold derivation script. It was also used for code-level debugging and document structuring. The research design, research questions, choice of methods, metric selection, threshold derivation rules, experimental protocol and all interpretation of results are the author's own work.

Two distinctions are stated explicitly. First, the derivation rules in Table 6 were specified

by the author; the derivation script implements those rules and computes their outputs from the measured data, and no threshold value was proposed or selected by an AI tool. Second, no AI tool generated, altered or fabricated any telemetry. All data derives from physical flights, and the raw session logs are published so that every reported figure can be recomputed independently. All AI-assisted code was reviewed, executed and validated by the author against real flights before use.

3.8 Threats to Results

Threats are organised by type, following standard practice in experimental software engineering (Wohlin et al. 2012). Risks to the project schedule are handled separately in Table 9.

Internal validity. Battery ageing, cell condition and ambient temperature vary between sessions and affect both depletion rate and thermal readings, so battery identifier and ambient temperature are logged per session and any outlier can be attributed rather than dismissed. More significantly, thresholds are derived from baseline flights on the same airframe later used for fault runs, which risks fitting the thresholds to that specific unit. Pooling across five baseline sessions rather than one reduces this, but it cannot be eliminated with a single available drone, and it constrains the strength of the RQ2.1 claim.

Ambient temperature. The temperature threshold is expressed as an absolute value and is therefore sensitive to the conditions under which the baseline was collected. All five baseline sessions were flown at an ambient temperature of 21 °C and their settled means fall between 61.4 and 62.8 °C. A held-out normal flight conducted in a room estimated to be 1 to 2 °C warmer settled at 64.0 °C, above every baseline session, while its battery depletion rate and mean acceleration magnitude remained within the baseline range. A baseline collected at a single ambient temperature cannot quantify that relationship, so the derived value transfers only to comparable conditions.

Positioning and ambient lighting. The aircraft holds position using a downward-facing camera and a time-of-flight sensor, and therefore depends on ambient lighting. A session attempted in a dimly lit room produced time-of-flight readings ranging between 21 and 133 cm against a stable 77 to 78 cm in adequate light, uncommanded lateral drift, refusal of relative-position commands, and a battery depletion rate approximately 2.5 times the baseline. Lighting is therefore controlled

alongside venue and ambient temperature, and the affected session was excluded.

External validity. The study uses one drone model in a predominantly indoor environment with a small number of sessions, so the specific threshold values are not transferable to other platforms. The Tello’s telemetry is also narrower than that of the platforms in much of the reviewed literature, providing no GNSS data and only coarse Wi-Fi quality information, which limits how directly the findings compare to GPS-equipped research platforms. What is claimed as transferable is the derivation *procedure* and the pipeline architecture, not the numbers, and that distinction is maintained throughout the reporting of results.

Construct validity. A composite health-status metric compresses a multi-dimensional condition into a single value, and whether it represents ”health” in a way an operator would accept is a modelling assumption rather than a measured fact. Detection latency is bounded below by the scrape interval and the `for:` hold duration, so the figures reported measure the latency of *this configuration*, not an intrinsic property of threshold-based detection. Both bounds are stated alongside every latency result. Defining fault onset from the first raw threshold crossing partially couples the onset definition to the threshold under test; the operator induction marker is kept as an independent check against this.

Reliability. All session logs, configuration files, alert histories and PromQL queries are archived and published, so every reported figure can be independently recomputed from the raw data. Tool versions are pinned. Where a session is excluded from analysis, the exclusion and its reason are reported rather than the session being silently dropped.

3.9 Summary

The methodology is one experimental design executed as two studies against a single artefact. Study 1 established that the pipeline can observe health state, using a pre-labelled dataset so that the pipeline itself was the only thing under test. Study 2 removes the labels and requires the same pipeline to determine health state from raw telemetry on physical hardware, with every threshold derived from the drone’s own measured baseline rather than assumed. Experimental research supplies the controlled conditions that make detection latency and false positive rate measurable; design science research supplies the structure for evaluating the pipeline as a contribution in its own right. Evaluation criteria for both studies were fixed before the corresponding data was collected, and the limits of what the results can support are stated in Section 3.8 rather than left

implicit.

Sustainability and reuse. All research artefacts are published in a public repository on submission (<https://github.com/Dmatt547/RAS-monitoring-thesis.git>): both exporters, `prometheus.yml`, `rules.yml` with the derived thresholds, the Grafana dashboard as exported JSON, the full baseline and fault-run telemetry logs, the PromQL queries used in analysis, and a README documenting the reproduction procedure end to end. The Study 1 dataset is not redistributed, as it is licensed CC BY-SA 4.0 (Ekanayakadev1k 2024); it is cited and linked instead, with the preprocessing script provided so the replay can be reconstructed. Tool versions are pinned in Table 2 and the deployment is containerised, so the pipeline can be rebuilt without manual environment configuration.

Every component of the stack is open-source, with no paid licence, subscription or cloud service in the dependency chain. That is a deliberate design property, since the gap addressed is the absence of monitoring approaches that survive without cloud infrastructure, and it means the artefact keeps working after the project ends and after any university account access ends. The exporter presents a platform-agnostic interface, so any system reporting numeric telemetry can replace the Tello without altering the Prometheus, alerting or dashboard layers. The derivation rules in Table 6 are stated in terms of metric behaviour rather than drone-specific fields. Prometheus’ multi-target scrape model also allows extension from one drone to a fleet without architectural change. These extensions are discussed in Section 7.1.

4 Prometheus-Based Health Monitoring Framework for Remote Autonomous Systems (RQ1)

4.1 Aim

RQ1 asks: *How can monitoring frameworks be used to assess and maintain the health of remote autonomous systems?* To answer this question, a working monitoring pipeline had to be designed and built that could collect, store, and evaluate health telemetry from a RAS. This section documents that implementation, reports the results of a controlled experiment performed with flight data replayed through the pipeline, and evaluates how well Prometheus meets the evaluation criteria established in Table 4. This study validates the monitoring architecture and establishes that fault detection can be performed in real time. Section 5 then extends it to a physical aircraft.

4.2 Implementation

The monitoring framework was set up as a pipeline with three different components. The first was a Python telemetry exporter, which read the telemetry CSV file row by row, then fed into a Prometheus time-series database, and then into a Grafana visualisation dashboard. Each component has a specific role in the design. Together, they form the designed artefact evaluated in this chapter.

4.2.1 Python Telemetry Exporter

The exporter (`ras_exporter.py`) is a Python script that replays drone telemetry data from a CSV file, row by row, generated from the Drone Telemetry Tampering Dataset v2 (Ekanayakadevilk 2024). The Drone Telemetry Tampering Dataset v2 is a public UAV telemetry dataset containing 190 rows of labelled flight records across six anomaly phases for tampering

detection and anomaly research. The version of `ras_exporter.py` used in testing was created with the help of Claude (Anthropic). The monitoring architecture, research design, and experimental decisions are my own work. The exporter reads each row sequentially at an interval of one second and publishes the corresponding telemetry values to Prometheus using a custom HTTP endpoint on port 8000 using the `prometheus_client` library. Because the dataset already contains labelled anomaly phases and health status values, the system can detect health conditions directly from the telemetry data without using separate threshold rules.

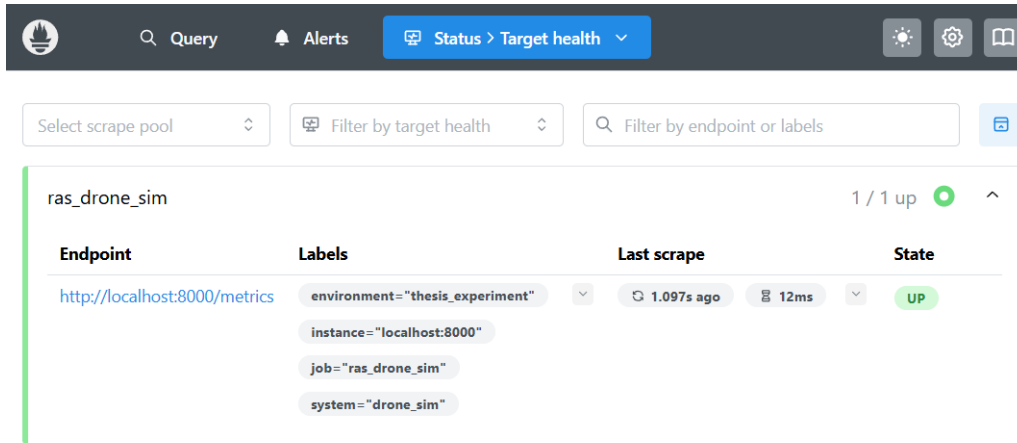


Figure 1: Prometheus Target health page confirming the RAS exporter endpoint is actively scraped with status UP.

Table 10 lists the eight metrics exposed by the exporter, along with their types, units, and descriptions.

Table 10: RAS Telemetry Metrics Exposed to Prometheus

Metric Name	Type	Unit	Description
ras_altitude_m	Gauge	m	Drone altitude
ras_speed_ms	Gauge	m/s	Drone speed
ras_heading_deg	Gauge	degrees	Drone heading
ras_latitude	Gauge	decimal	GPS latitude
ras_longitude	Gauge	decimal	GPS longitude
ras_anomaly_label	Gauge	Binary	0 = normal, 1 = anomaly
ras_health_status	Gauge	Binary	1 = healthy, 0 = fault phase
ras_fault_events_total	Counter	Count	Total healthy → fault transitions

The `ras_health_status` metric reflects the ground-truth health label from the dataset, transitioning from 1 to 0 when the current row belongs to a fault phase. The `ras_fault_events_total` counter increments each time the pipeline detects a transition from a healthy state to a fault state, providing a running count of fault events over the session.

4.2.2 Prometheus

Prometheus was configured to scrape the exporter endpoint (`http://localhost:8000/metrics`) at a 15-second interval. All scraped metrics are stored as time-series data and made available for querying via

PromQL. The Prometheus configuration is defined in `prometheus.yml`, which is published with the other research artefacts as described in Section 3.9. Figure 1 confirms the exporter endpoint is successfully registered and actively scraped.

4.2.3 Grafana

Grafana was connected to Prometheus as a data source and used to build a monitoring dashboard providing a single-pane view of RAS health across all telemetry metrics. This closely follows the observability requirements found in the literature review (Ben Hafaiedh & Ben Slimane 2022).

4.3 Experimental Setup

4.3.1 Dataset

The dataset used in the experiment is the Drone Telemetry Tampering Dataset v2, sourced from Kaggle. The dataset consists of 190 rows of DJI flight log records, categorised across six different phases representing normal and anomalous flight behaviour. Each row has the telemetry values for altitude, speed, heading, GPS coordinates with timestamp, a pre-labelled anomaly flag and health status.

4.3.2 Software Configuration

- **Python 3.x** with `prometheus_client` library. This is the telemetry exporter.
- **Prometheus v2.x** controls metrics scraping and storage, with scrape interval set to 15 seconds.
- **Grafana** dashboard visualisation, connected to Prometheus via HTTP data source
- **Exporter endpoint:** `http://localhost:8000/metrics`

4.3.3 Dataset Phases

This data set has six labelled phases that are played back through the pipeline in order. Table 11 outlines each phase, the type of anomaly, and the health status assigned by the dataset.

Table 11: Dataset Phases Replayed Through the Monitoring Pipeline

Phase	Description	Health Status
normal_operation	Clean flight data, no anomalies	Healthy (1)
data_injection	Tampered or injected data values	Fault (0)
altitude_anomaly	Abnormal altitude readings	Fault (0)
speed_anomaly	Abnormal speed readings	Fault (0)
heading_anomaly	Abnormal heading readings	Fault (0)
combined_fault	Multiple simultaneous anomalies	Fault (0)

The pipeline processes rows one second at a time, so it will take about 190 seconds for the entire 190 row dataset to complete. Prometheus scrapes the exporter every 15 seconds, which gives multiple data points for every phase.

4.4 Results

4.4.1 Prometheus Metric Collection

The Prometheus UI confirmed that metrics were successfully collected during the entire experiment. The `ras_health_status` metric is shown in Figure 2 during the experiment for a period of 15 minutes in the Prometheus expression browser. The health state of the RAS is shown on the y-axis, where 1 is normal operation and 0 indicates a fault condition. The x-axis is time of the experiment, from 05:55 to 06:10 on 17 May 2026.

In the first half of the graph the metric is constant at 1.00 which corresponds to the normal operating phase of the dataset, where no faults exist. The value drops to 0.00 at approximately 06:00, indicating the first fault phase. The drop has a step-like shape, reflecting the pull-based scrape model of Prometheus, which records changes at fixed 15-second intervals rather than

continuously. The metric then becomes 0 for the rest of the visible period, signifying that the fault condition is ongoing and not a brief temporary spike.

The series label below the graph `ras_health_status{environment="thesis_experiment", instance="localhost:8000", job="ras_drone_sim", system="drone_sim"}` confirms that Prometheus is correctly linking the metric with the exporter instance and experiment setup. This proves that the monitoring system is not only collecting and storing the health data in real-time but also preserving the label information required for filtering, querying, and alerting in future experiments.

4.4.2 Grafana Dashboard

Figure 3 shows the Grafana dashboard during a live experiment run, displaying all eight metrics at the same time across a 15-minute period with a 5-second auto-refresh interval. The dashboard is organised into six panels, with each panel showing a different aspect of the RAS telemetry data.

The Altitude (m) panel (top left) shows the drone altitude oscillating between approximately -100 m and 100 m during the first part of the experiment, before settling down after 16:00, indicating the transition to a long-lasting fault state, where the altitude values stay the same. The Speed (m/s) panel (top right) stays close to zero during normal operation, and then sharply spikes to around 4 m/s near 16:00, which corresponds to the speed anomaly phase in the dataset. The Heading (degrees) panel (middle left) shows fluctuations between -126° and -132° during the anomaly, compared to the more stable readings seen during nominal operation.

The clearest indication of a fault is the System Health Status panel (middle right). The metric remains at 1 (healthy) until 16:00 and then drops sharply to zero, the same as the change in Prometheus browser. This is supported by the Anomaly Label panel (bottom left) where the value changes from 0 (normal) to 1 (anomaly) at the same time and stays there for the rest of the window. The Total Fault Events counter (bottom centre) shows 1, which means there was one transition from healthy to fault state during the experiment period. Finally, the Current Altitude gauge (bottom right) shows a live reading of 68.3 m at the time of the screenshot, showing the dashboard's capability to display real-time values alongside time-series panels.

The overall dashboard shows all parts of the monitoring system (data collection, storage, visualisation) are working correctly and the transition from a fault can be identified across multiple related metrics at the same time, not just a single metric.

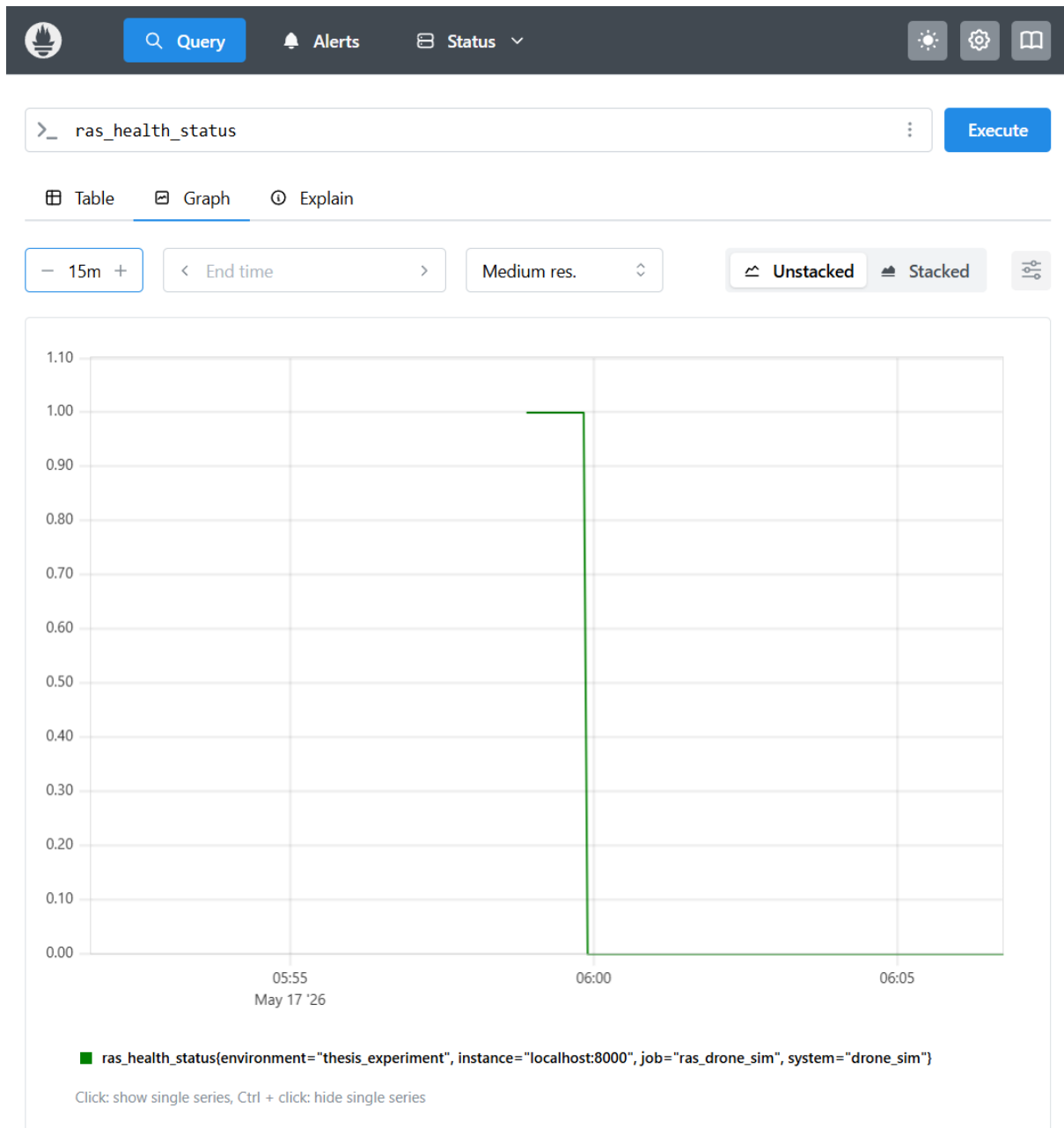


Figure 2: Prometheus browser showing `ras_health_status` transitioning between 1 (healthy) and 0 (fault) across dataset phases.

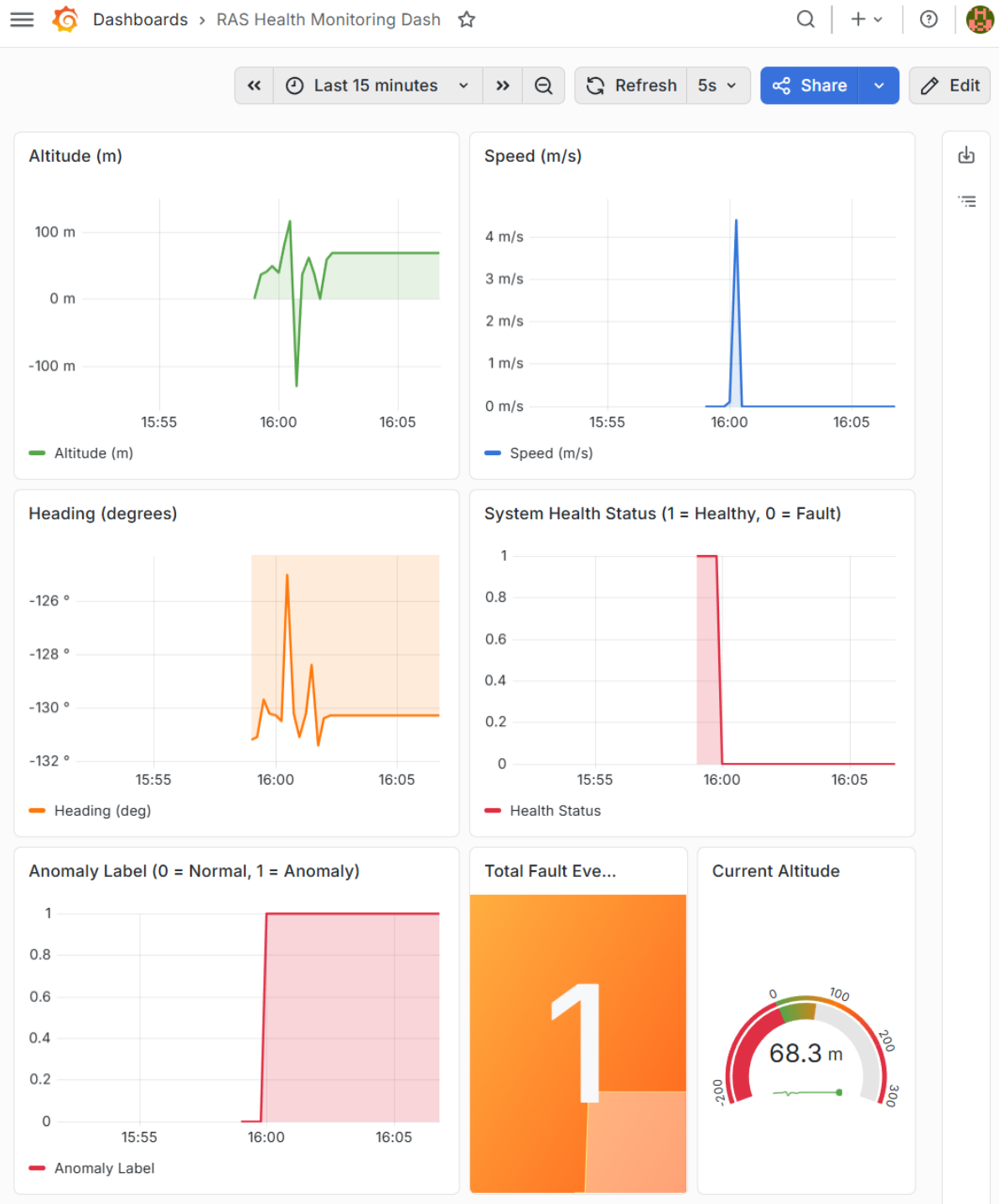


Figure 3: Grafana dashboard displaying real-time RAS telemetry replayed from the DJI flight dataset. Fault phase transitions are visible as step changes in the health status panel.

4.4.3 PromQL Queries Used

The following PromQL queries were used during the experiment to monitor and evaluate system state:

```
# Real-time health status (1 = healthy, 0 = fault phase)
ras_health_status
```

```
# Anomaly label from dataset (0 = normal, 1 = anomaly)
ras_anomaly_label
```

```
# Drone altitude over time
ras_altitude_m
```

```
# Drone speed over time
ras_speed_ms
```

```
# Total fault phase transitions detected
ras_fault_events_total
```

```
# Rate of fault events over 5-minute window
rate(ras_fault_events_total[5m])
```

4.4.4 Evaluation Criteria Results

Table 12 summarises the outcome against each evaluation criterion defined in Table 4.

Table 12: Evaluation Criteria Results

Criterion	Target	Result	Met?
Metric coverage	All key health signals captured in Prometheus	8 metrics captured including flight telemetry, anomaly label, health status, and fault counter	Yes
Detection latency	Fault conditions flagged within 10 seconds	All fault phase transitions detected within one 15-second scrape cycle	Partial
False positive rate	Alerts triggered during normal operation minimised	Zero false positives observed during the normal_operation phase	Yes
Dashboard completeness	All metrics and alert states visible in Grafana	Dashboard displays all eight metrics with health status and fault event count	Yes

4.5 Analysis

The experiment showed that the Prometheus monitoring pipeline successfully collected, stored and evaluated drone telemetry data, detecting all transitions into a fault phase without any false positives during normal operation. These results directly answer RQ1 by validating that Prometheus is a practical and useful framework for monitoring the health of a remote autonomous system.

Three evaluation criteria out of four were fully satisfied. The one partial result, detection latency, again shows the 15 second scrape interval, which bounds how quickly any metric change is observed by Prometheus. This is a configurable parameter and reducing the interval solves this at the cost of higher storage overhead (Boncea & Bacivarov 2016). An interesting finding was that the exporter updated metric values every second, so telemetry changes appeared almost immediately at the metrics endpoint. This means the main delay came from the Prometheus scrape cycle rather than the pipeline itself.

Using a structured, pre-labelled dataset rather than purely random values makes the pipeline evaluation more reliable. The Drone Telemetry Tampering Dataset v2 contains labelled anomaly phases, meaning the fault transitions tested are based on plausible behaviour instead of random parameters. The pipeline’s ability to accurately reflect the dataset’s health labels through `ras_health_status` and increment `ras_fault_events_total` on each fault transition is proof that the monitoring architecture is functioning correctly under real-world conditions.

`ras_anomaly_label` is a good secondary signal to complement `ras_health_status`. `ras_health_status` describes the overall health state at the phase level whereas `ras_anomaly_label` marks specific data points that are found to be anomalous in the dataset. The combination of these two metrics provides a system-level and data-point-level view of RAS health, which follows the multi-layer monitoring approach described in the literature (Rafique et al. 2024).

One limitation of this study is that the health label came directly from the dataset rather than being calculated by the pipeline from raw telemetry values using pre-defined thresholds. This is acceptable for validating the pipeline, but it leaves untested the case where fault conditions must be determined solely from raw sensor readings. That limitation is what Section 5 addresses, where a physical DJI Tello streams live telemetry and Prometheus alerting rules perform fault detection without any pre-labelled data.

4.6 Answer to RQ1

RQ1 raised the question: *How can monitoring frameworks be used to assess and maintain the health of remote autonomous systems?* The results shown in this section demonstrate that a Prometheus-based monitoring framework is capable of continuously collecting, and evaluating health telemetry from a RAS. It is also able to accurately monitor fault phase transitions in real time using structured drone telemetry data. The designed pipeline that includes a Python telemetry exporter, Prometheus and Grafana was able to capture all defined health signals, correctly identify all fault phase transitions with no false positives, and show system state in a complete and straightforward dashboard.

The main mechanisms enabling this are the pull-based scrape model of Prometheus, which provides regular metric collection, and the use of a combined health status metric that brings the overall system state into a single easy-to-monitor value. These design patterns carry directly into the physical drone integration reported in Section 5, where the exporter is connected to a live DJI Tello using the Tello SDK and Prometheus alerting rules replace the pre-labelled health values of the dataset. The evidence produced here validates the pipeline architecture and provides the foundation for that evaluation.

5 Autonomous Threshold-Based Fault Detection on a Physical Remote Autonomous System (RQ2)

5.1 Aim

RQ2 asks: *How can system metrics such as battery level, connectivity and performance be used to detect faults in remote autonomous systems?* Study 1 showed that the pipeline can observe health state, but that state came from a dataset label rather than from the telemetry. This chapter removes the dependency. The same pipeline is connected to a physical aircraft, every threshold is derived from the aircraft's own measured baseline, and Prometheus rules decide health from raw readings with no labels available to them.

Whatever their form, remote autonomous systems report health through four recurring signal types: stored energy, thermal state, control stability, and link liveness. An aerial vehicle reports these as battery percentage, onboard temperature, attitude and contact with the ground station, while a ground robot or an underwater vehicle reports the same four through different sensors. They matter here because each behaves differently over a mission, and Section 5.2.2 shows that this behaviour is what decides the rule used to set a threshold. The platform chosen to study them is a DJI Tello quadrotor, because it exposes all four over a documented SDK and can be flown repeatedly and safely indoors. It is a low-cost aircraft rather than a field platform, and Section 5.5.2 reports where its instrumentation falls short.

The chapter turns on the difference between collecting telemetry and calibrating a decision rule against it. Collection is largely solved by existing observability tooling, as Study 1 confirmed. Deciding what counts as abnormal is not, because the answer is a property of the platform rather than of the monitoring stack. Study 2 therefore treats calibration as the object of study, and states the procedure before reporting any result so that every threshold can be traced back to the data that produced it.

The experiment runs in three phases, measured against the criteria fixed in Table 7. Phase 1 derives the thresholds and answers RQ2.1, Phase 2 tests whether the rules detect induced faults for RQ2.2, and Phase 3 quantifies detection latency and false positive rate for RQ2.3.

5.2 Design

Study 2 needed a system that could fly a repeatable flight, record what the aircraft reported while it flew, decide whether that reading was normal, and make the decision reviewable afterwards. This section sets out the structure chosen to do that, and the reasoning behind it. The tools that realise the structure are described in Section 5.3.

5.2.1 Monitoring Architecture

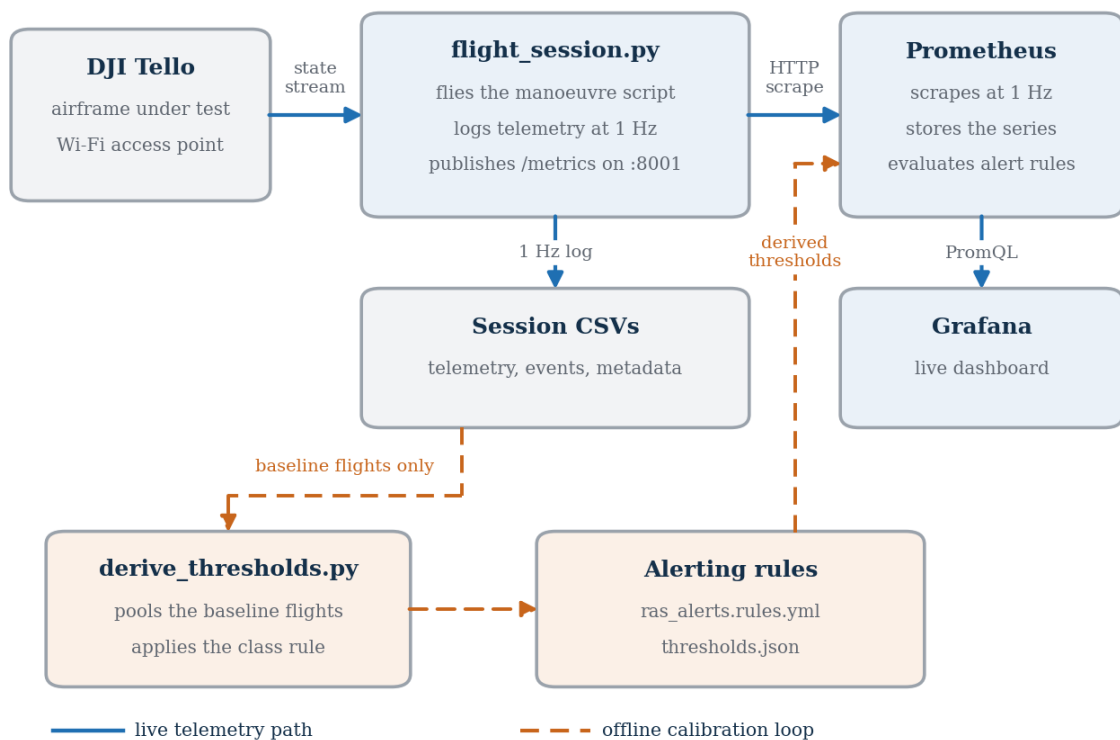


Figure 4: Architecture of the Study 2 monitoring system. The live telemetry path runs from the aircraft to the dashboard during a flight. The calibration loop runs offline between flights and decides what the alerting rules contain.

Figure 4 shows the two paths the design is built from, and the separation between them is the main design decision in this study.

The live path runs left to right while the aircraft is flying. The Tello broadcasts its state over

Wi-Fi at 1 Hz, the session harness receives that stream and does three things with it at once, and Prometheus scrapes the harness at the same rate and evaluates the alerting rules against whatever it has just collected. Grafana reads from Prometheus rather than from the aircraft, so the dashboard is a view of the stored series and not a second consumer of the telemetry. Nothing on this path makes a judgement of its own. Each component moves data or applies a rule that was decided elsewhere.

The calibration loop runs offline, between flights, and is where the judgements are made. Every session is written to disk as three CSV files, but only the baseline flights feed `derive_thresholds.py`. That script pools those flights, applies the derivation rule appropriate to each metric's behaviour, and writes the alerting rules file that Prometheus loads. The procedure it follows is given in Section 5.2.2.

Separating the two paths is what makes the thresholds traceable. Because no threshold is written by hand into the rules file, and because the only input to the calibration loop is measured flight data, every value Prometheus evaluates can be traced back to the flights that produced it. It also means the two halves can fail independently: a fault in the live path shows up as missing data, while a fault in the calibration loop shows up as a rule that fires when it should not, and Section 5.8 reports four occasions where the second happened.

Two further decisions follow from the platform rather than from preference. The harness is a single process because the Tello accepts one SDK client at a time, so flying, logging and publishing cannot be split across separate programs competing for the same UDP ports. Flights are scripted rather than hand-flown, because three repetitions of a fault must be the same fault for the repetitions to mean anything, and a human pilot would introduce variation into every measurement.

5.2.2 The Calibration Procedure

All of Study 2 follows one four-step loop, shown in Figure 5. The steps are measure, derive, encode and test. First the aircraft is measured in normal flight. A threshold is then derived from that measurement, using a rule chosen in advance. The threshold is encoded as a Prometheus alerting rule. Finally the rule is tested on normal flights that played no part in deriving it. That last step is what separates the procedure from simply picking a number, because a threshold that has never been tested on held-back normal data cannot be shown to be free of false alerts.

```

INPUT   Baseline flights, flown under normal operation
        Held-back flights, also normal, kept out of the derivation
OUTPUT  One alerting rule for each usable metric

STEP 1  MEASURE NORMAL OPERATION
1  pool every in-flight sample from the baseline flights
2  drop samples taken before take-off or after landing,
3      because those are not normal operation

STEP 2  DERIVE A THRESHOLD FOR EACH METRIC
4  for each metric:
5      decide how the metric behaves, then apply its rule

6      settles towards an operating point:
7          99th percentile of the last 90 s of each flight
8      varies around a steady level:
9          99th percentile of its 5 s rolling deviation
10     falls steadily by design:
11         failsafe point, plus the warning time needed
12         at the worst rate observed
13     is either present or missing:
14         no threshold, because absence is the fault

STEP 3  ENCODE EACH THRESHOLD AS AN ALERTING RULE
15  add a hold, so one stray sample cannot raise an alert
16  add an airborne gate if the metric only applies in flight

STEP 4  TEST ON THE HELD-BACK FLIGHTS
17  for each held-back flight:
18      if any rule raises a sustained alert on it:
19          the metric was classified wrongly in Step 2,
20          or Step 1 did not cover this kind of flight
21          fix the cause and start again from Step 1

22  return the rules

```

Figure 5: The threshold calibration procedure applied throughout Study 2

Two steps do most of the work. Step 2 decides which statistic is meaningful for a given metric, and Section 5.5.3 shows that getting this wrong produces a threshold that is not merely

imprecise but invalid. Step 4 decides what happens when a healthy flight raises an alert. The threshold is not simply moved, because that would leave the cause unexamined. The alert is taken instead as a sign that the metric was classified wrongly, or that the baseline data did not cover this kind of flight, and the procedure starts again. This happened three times during the study, and each case is reported.

5.3 Implementation

Study 2 required three capabilities that Study 1 did not: repeatable physical flights, thresholds computed from measured data, and alert timings that can be recovered after a run has finished. Each is provided by a separate tool, and all are published in the project repository.

5.3.1 Session Harness

`flight_session.py` is the instrument used for every physical flight. A single process performs three jobs, because the Tello accepts one SDK client at a time and a second client competing for the same UDP ports would prevent both from working.

1. It flies a fixed manoeuvre script consisting of take-off, hover, translation, rotation, hover and land, so that variation between sessions reflects the platform rather than the pilot.
2. It records raw telemetry to a CSV file at 1 Hz and tags every sample with the manoeuvre step that was executing when the sample was taken.
3. It publishes the same readings to Prometheus on port 8001.

Manual piloting was considered and rejected. Three repetitions of a fault must be the same fault for the repetitions to be meaningful, and hand-flown runs would introduce operator variation into every measurement. The single-client constraint also rules it out in practice, since flying from the mobile application would occupy the connection the exporter requires.

The harness writes a separate event log containing manoeuvre boundaries, stop reasons and operator markers. Pressing ENTER during a flight records t_{induce} , the moment the operator began the injection. That marker is recorded for completeness but is not used to compute detection latency, for the reason given in Section 5.6.4.

Four modes are provided. The `baseline` mode flies the Phase 1 script. The `battery` mode disables the harness's own landing so that the aircraft's low-battery failsafe ends the run, since that failsafe is the object of measurement rather than a constraint to be avoided. The `thermal` mode flies a vertical oscillation profile after a hot start. The `link` mode holds a stationary hover while the ground station's wireless interface is disabled. The two fault modes are described in Section 5.3.2, because both differ from the mechanisms planned in Section 3.5 and the reasons for the change are themselves results.

A pre-flight check refuses to begin a session on a battery below 25 %, since a run that ends through exhaustion part-way through the manoeuvre script cannot be compared with the others. A second check rejects an onboard temperature above 60 °C. That check is skipped in thermal mode, where a hot airframe is the experimental condition rather than a fault in the setup.

5.3.2 Fault Injection Mechanisms

Two of the three fault mechanisms specified in Section 3.5 did not produce the intended condition on this platform and were replaced. The replacements are described here, and the evidence that forced each change is reported with the corresponding Phase 2 results.

Battery depletion was injected as planned, by flying until the firmware's own low-battery failsafe engages.

Thermal stress was planned as sustained aggressive manoeuvring. That mechanism does not work, because flight cools this aircraft: rotor airflow removes heat from the sensor board faster than any commanded manoeuvre adds it, and an applied external heat source was equally ineffective once the rotors were turning. The mechanism used instead is approximately eight minutes of powered ground idle followed by launch, which produces a hot start that decays toward the normal operating point. The fault this represents is insufficient thermal recovery between sorties, which is a realistic operational condition for an aircraft flown repeatedly.

Link degradation was planned as increasing physical separation between aircraft and ground station. That mechanism does not work either, because the platform's transmit range exceeds the available indoor test area, so the link could not be degraded within the flight envelope. The mechanism used instead is disabling the ground station's wireless interface during a stationary hover, which removes the link at a known instant without moving the aircraft.

5.3.3 Metric Set and Platform Constraints

A monitored metric is only useful if it carries a signal that changes when the system is unhealthy. Each of the four signal types named in Section 5.1 is therefore mapped to the field that reports it on this aircraft, and every field is checked against measured data before it is trusted. Table 5 lists the fields the Tello exposes.

Baseline collection established that several of them behave differently from what their SDK field names imply, and those findings are reported in Section 5.5.2. The metric set carried through to the alerting rules is battery percentage, onboard temperature, attitude and target liveness, which are the four signal types above. Barometric height, reported height, wireless signal-to-noise ratio and the velocity fields are logged and reported but carry no rule, for reasons given in Section 5.5.2.

5.3.4 Alerting Rules

Fault detection is performed entirely by Prometheus alerting rules evaluated against the raw metrics. No metric published by the exporter is a health label, and there is no equivalent of the `ras.health.status` metric used in Study 1. Every rule carries a hold, expressed as the `for` clause, so that a single sample crossing a threshold cannot raise an alert. The hold is 5 s for the three threshold rules and 2 s for the liveness rule, which needs no averaging because absence of data is unambiguous. The rules are presented in Section 5.5.3 together with the values derived for them.

5.3.5 Analysis Tooling

Three further scripts make the results reproducible from the archived logs.

- `check_session.py` validates a session immediately after landing. It checks sample coverage against the expected 1 Hz, gaps between samples, battery monotonicity, SNR coverage, manoeuvre completeness and metadata. A session that fails is re-flown rather than retained.
- `derive_thresholds.py` pools the baseline sessions, applies the derivation rules given in Table 6, and writes both the Prometheus rules file and a machine-readable copy of the same values.

- `collect_alerts.py` recovers alert firing times from the `ALERTS` series that Prometheus maintains for its own alert state, computes threshold-crossing times from the telemetry log, and reports detection latency. Both calculations read the machine-readable threshold file, so the onset calculation and the alerting rules cannot diverge.

Because Prometheus retains the `ALERTS` series in its own time-series database, alert state can be recovered after a flight has ended by re-querying the stored window. Every figure in this chapter is produced from stored data rather than captured live during a flight, and two runs whose alert timings were initially missing were recovered this way without re-flying them.

5.4 Experimental Setup

5.4.1 Platform and Environment

All sessions used a single DJI Tello, serial number 0TQZGAJED007GD, flown indoors at a private residence at an ambient temperature of 20 to 22 °C. One airframe was used throughout so that the thresholds and the fault runs refer to the same unit. The consequences of that decision are addressed in Section 3.8.

Two batteries, designated B1 and B2, were alternated between sessions so that battery identity would not coincide with session order. Prometheus, Grafana, the harness and the exporter all ran on a single host, so every timestamp comes from one system clock and clock synchronisation between devices cannot introduce measurement error. The scrape interval and the rule evaluation interval were both set to 1 s.



Figure 6: The test environment during a live session, with the Tello airborne and the harness running on the ground station. Depth of field has been added in post-processing to separate the aircraft from the background.

5.4.2 Session Protocol

Each baseline session ran for four minutes and began on a fully charged battery, with the aircraft returned to ambient temperature beforehand. Four-minute sessions sample only the upper region of the discharge curve. The lower region, approaching the automatic low-battery landing, is characterised instead by the Phase 2 depletion runs, which continue until that failsafe engages.

Lighting was controlled alongside ambient temperature. The Tello holds position using a downward-facing camera and a time-of-flight sensor, and a session attempted in a dimly lit room produced erratic readings and uncommanded drift. Clearing two metres of plain floor and lighting the room removed the problem for later sessions. That result is reported in Section 3.8.

Two deviations from this protocol are declared rather than left for a reader to find. All three link runs used battery B1 where the runsheet specifies alternation, so battery identity is

confounded with the link fault class. The thermal runs deliberately began on part-discharged cells, at 57, 84 and 48 %, because the mechanism requires a warm cell that has already been powered for several minutes. The consequence of the second deviation is visible in the results and is reported in Section 5.6.5.

5.4.3 Experimental Runs

Table 13 lists every run in the study. Eighteen sessions were accepted and a further nineteen were rejected; the rejected sessions are retained in the repository with the reason for exclusion recorded, following the reliability commitment stated in Section 3.8.

Table 13: Study 2 Experimental Runs

Phase	Runs	Mode	Purpose	Sub-question
1	BASE-01-05	baseline	Characterise normal operation and derive thresholds	RQ2.1
–	FP-01-03	baseline	Normal flights held out from derivation, for the false positive rate	RQ2.3
–	WARM-01	baseline	Healthy thermal control, flown immediately before the thermal runs	RQ2.2
2	FAULT-BAT-01-03	battery	Flight continues until the low-battery failsafe engages	RQ2.2, RQ2.3
2	FAULT-THERM-01-03	thermal	Launch after a powered ground soak, without a cool-down interval	RQ2.2, RQ2.3
2	FAULT-LINK-01-03	link	Ground station wireless interface disabled during a stationary hover	RQ2.2, RQ2.3

5.5 Phase 1 Results: Baseline Characterisation (RQ2.1)

5.5.1 Session Summary

Five baseline sessions were flown and all five passed validation, producing 1274 pooled in-flight samples. Table 14 summarises them. Sample coverage was 100 % in every session and no gap between consecutive samples exceeded 1.1 s, so the distributions reported below are computed from complete data. In-flight here means every sample whose manoeuvre tag is not preflight, takeoff, land or postflight, because those four steps are not normal operation and including them would place ground readings in a distribution intended to describe flight.

Table 14: Phase 1 Baseline Sessions

Session	Battery	Ambient (°C)	Duration (s)	Battery span (%)	Depletion (%/min)	In-flight temp (°C)
BASE-01	B1	21	271	100–66	7.53	56–63
BASE-02	B2	21	270	88–40	10.67	57–62
BASE-03	B1	21	273	100–66	7.47	56–62
BASE-04	B2	21	273	92–45	10.33	61–65
BASE-05	B1	21	275	100–69	6.76	58–63

5.5.2 Platform Behaviour

Baseline collection produced six findings about the platform, each of which changes how the metric concerned can be used. They are reported here rather than in the analysis because they form part of the answer to RQ2.1: establishing which metrics represent operational health requires knowing how those metrics actually behave, and on this platform several of them do not behave as their field descriptions suggest.

Onboard temperature settles toward an operating point. Temperature does not vary about a fixed mean during a session. It converges toward a steady operating value of approximately 62 °C, and the direction of change depends on where the aircraft starts. Sessions beginning at ambient warmed from 55–58 °C to roughly 62 °C over four minutes. Session BASE-04, which began at 65 °C because it followed an earlier flight, cooled to the same point instead. Table 15 shows the progression, reported as the mean of the in-flight samples falling in each interval because the field itself is integer-valued and a single sample would not show the trend.

Table 15: Onboard Temperature by Elapsed Flight Time (°C), Mean per Interval

Session	Start	0–30 s	30–60 s	60–120 s	120–180 s	180–240 s	240–300 s
BASE-01	55	57.7	59.3	61.1	61.9	62.4	62.5
BASE-02	58	57.8	58.6	59.8	60.6	61.5	61.4
BASE-03	56	57.5	58.5	59.9	61.0	61.5	61.9
BASE-04	65	63.0	62.1	62.3	62.6	62.8	62.7
BASE-05	58	59.2	60.5	61.3	62.0	62.2	62.2

The **Start** column is the first sample of the session, recorded on the ground before take-off. Every session except BASE-04 is below its eventual operating point at that moment, and BASE-04 is above it, which is the clearest single demonstration that the quantity converges rather than fluctuating.

Two consequences follow. The first concerns the assumed threshold carried into Study 2. A ceiling of 60 °C, taken from industrial motor-monitoring practice, sits below the operating point

the aircraft reaches in normal flight, so the rule fires on every healthy session once the airframe has warmed. That is the false positive observed during preliminary testing.

The second concerns the derivation rule, and is treated in Section 5.5.3. Because a four-minute session spends most of its samples in the warm-up transient rather than at the operating point, a percentile taken over the whole session does not describe steady operation.

The barometer has no stable inter-session reference. The absolute barometric reading re-zeroes after each power cycle. Across the accepted sessions, all flown on the same floor in the same room, the median absolute value spans approximately 167 cm, from -65.2 to 101.7 cm, despite the aircraft's ground clearance remaining unchanged. Within an individual session, the reading is stable, ranging from 0.37 to 2.07 cm. The metric is therefore precise within a session, but its value has no meaning after a power cycle. As a result, a threshold expressed in its units cannot be compared across sessions. An earlier iteration derived and tested a drift-based rule, which measured deviation from the metric's own rolling mean, as discussed in Section 5.5.3. This rule was retired, and no barometric rule is carried into Phase 2.

Reported height is a derived quantity, not a measured one. The median value returned by `get_height()` ranged from 0 to 70 cm across sessions, while time-of-flight ground clearance remained between 71 and 96 cm over the same sessions. In FP-03, the two values differed by almost one metre within a single session: the median reported height was 0 cm, while the median time-of-flight clearance was 96 cm. The aircraft maintained altitude, but the derived height value drifted.

Height and barometer are therefore treated as drift indicators, while time-of-flight is used as the altitude signal. This reverses the roles assigned to these fields in Table 5. The reassignment is based on measured behaviour rather than the SDK documentation. Time-of-flight itself proved reliable, returning values up to 197 cm with no saturation or dropout.

Battery identity dominates the depletion rate. Battery B1 depleted at a mean rate of $7.25\% \text{ min}^{-1}$ across three sessions, with individual runs at 7.53 , 7.47 and $6.76\% \text{ min}^{-1}$. Battery B2 depleted at $10.50\% \text{ min}^{-1}$ across two sessions, at 10.67 and $10.33\% \text{ min}^{-1}$. The two distributions do not overlap. B2 also never charged above 92% while B1 reached 100% in every session, which gives a second and independent indication of a degraded cell. A single depletion threshold cannot describe both batteries, so the derivation in Section 5.5.3 takes the worse of the two. Alternating batteries between sessions is what made the difference visible.

Wireless SNR carries no usable variance. SNR returned the same value, 90, for every valid in-flight sample in the baseline corpus. Because the value never changed, no threshold could be derived from it. A percentile of a constant value is still that same value.

SNR is also available only through a command-channel query, rather than the broadcast state stream. Sampling it at 1 Hz would compete with flight commands, so it was polled every 5 s. The most recent value was carried forward between polls, and its staleness was recorded alongside it.

Velocity fields are uninformative at the sampling rate used. All three velocity components read zero simultaneously in 87.6 % of airborne baseline samples, including during commanded translation, because each movement completes between samples. The manoeuvre is instead visible in attitude, where pitch reaches 8° during translation compared with 2° in hover.

5.5.3 Derived Thresholds

Table 16 lists the threshold for each monitored metric, its metric class from Table 6, and the rule used to derive it. Every threshold comes from a stated rule applied to the pooled baseline data. None is assumed.

Table 16: Thresholds Derived from Phase 1 Baseline Data

Alert	Metric class	Hold	Value	Derivation
DroneLowBattery	Monotonic	5 s	31 %	Firmware auto-land point of 10 % plus two minutes of lead time at the worst observed depletion rate of $10.50\% \text{ min}^{-1}$
DroneOverTemperature	Settling	5 s	64 °C	99th percentile of the settled region, taken as the final 90 s of each session, applied in the high direction only and gated on <code>ras_tof_cm > 20</code>
DroneUnstableAttitude	Variance	5 s	3.87 pitch, 1.20 roll	99th percentile of the 5 s rolling standard deviation
DronePollFailure	Liveness	2 s	n/a	Binary. Missing data is itself the fault condition and no value is derived

The temperature threshold requires one clarification. The 99th percentile of the pooled settled data is 63.5 °C, but the rules file rounds this to 64 °C. The temperature field is reported in whole degrees, so a fractional threshold could not be crossed. Both values are included in the machine-readable threshold file, making the rounding explicit.

Two parts of the temperature rule require additional conditions. First, the rule is evaluated

only while the aircraft is airborne, using `ras_tof_cm > 20`. The threshold was derived from in-flight samples, but 12 % of healthy ground samples in the baseline exceed it, for the reason described in Section 5.5.2. Without this airborne guard, the alert would fire before every take-off.

Second, the temperature rule applies only in the high direction. A two-sided rule using the 1st percentile, 57 °C, would alert in the first few seconds of healthy flight while the aircraft is still below its operating temperature. For example, BASE-01 recorded 56 °C for six consecutive seconds during this period. This exceeds the 5 s hold and would have raised an alert. As this platform has no identified under-temperature fault mechanism, a lower threshold would add false positives without providing useful detection. Table 6 supports this by specifying the percentile direction for each metric.

The temperature threshold is derived from the settled part of each session, rather than the full session, as established in Section 5.5.2. Using every in-flight sample gives a threshold of 63 °C. This falls within the 62 to 64 °C range seen once the aircraft has settled, because warm-up accounts for most samples in a four-minute session and lowers the percentile. When applied, this lower threshold produced a sustained alert on a held-out healthy flight.

Restricting the derivation to the final 90 s of each session raises the threshold by one degree. No baseline or held-out flight then produced a sustained crossing. Two held-out flights did reach 65 °C briefly, but the hold prevented these short excursions from becoming alerts. The difference between an exceedance and an alert is quantified in Section 5.6.1.

The same reasoning applies to battery level. A metric that changes by design throughout a session should not use a percentile of its full distribution, because the trend dominates the result. Battery level trends downward, while temperature moves towards an operating point. Neither is stationary in the way assumed by the bounded-metric rule.

An earlier version of the procedure included a barometric drift rule. It measured deviation from the metric's own 60 s rolling mean, used a threshold of 0.31 cm, and evaluated only once the full window contained airborne samples. The rule was retired before Phase 2 for the reason given in Section 5.5.2: the barometric reference re-zeroes after every power cycle, so alerts cannot be meaningfully compared between sessions. A drift-based rule hides this limitation rather than resolving it.

The rule remained loaded in Prometheus while Phase 2 was flown. As a result, `DroneBarometricDrift` appears in the historical `ALERTS` series for these runs, but it is excluded from all results reported in this chapter.

5.5.4 Assumed and Derived Thresholds Compared

Table 17 sets the thresholds assumed before baseline collection against those derived from it. Both configurations are published in the repository so that the comparison can be verified.

Table 17: Assumed and Derived Thresholds Compared

Condition	Assumed	Derived	Consequence of the assumed value
Low battery	$< 15\%$	$< 31\%$	Leaves under 30 s of lead time before the failsafe at the observed depletion rate
Temperature	$> 60^{\circ}\text{C}$	$> 64^{\circ}\text{C}$, airborne only	Fires on every healthy flight in the corpus once the airframe has warmed
Attitude	$ \text{pitch} \text{ or } \text{roll} > 30^{\circ}$	5 s rolling standard deviation > 3.87 or 1.20	Baseline attitude never exceeds 8° , so the rule cannot fire and detects nothing

The two failures are not equivalent, and the difference between them is worth stating. The assumed temperature rule produced a visible false positive and was noticed during preliminary testing. The assumed attitude rule produced no alerts at all, which is indistinguishable from correct operation until a fault occurs and is not detected. An assumed threshold can therefore fail in either direction, and only one of the two failure modes is self-announcing.

5.6 Phase 2 Results: Fault Detection (RQ2.2)

Nine fault runs were flown, three in each fault class. Every run was validated after landing, and alert state was recovered from the Prometheus ALERTS series for all nine.

5.6.1 Held-Out Normal Flights

Three normal flights were held out from threshold derivation and flown under the baseline protocol, giving 793 s of airborne time, or approximately 13.2 min. No rule raised a sustained alert in any of the three. Adding the five derivation sessions brings the healthy dataset to approximately 35.1 min of airborne time, again with no sustained alert.

The same eight flights did produce 33 short threshold crossings that never became alerts,

13 on temperature and 20 on attitude. Crossings are counted here exactly as the deployed rules evaluate them, so the temperature count is restricted to samples satisfying the airborne gate and the attitude count is not, because the attitude rule carries no gate. Eight of the 13 temperature crossings occurred in a single session, FP-02. Every one of the 33 was shorter than its rule's hold and was therefore suppressed. Without the hold the healthy dat would have produced 33 alerts rather than none, so the hold is what makes the derived thresholds usable rather than merely correct.

5.6.2 Battery Depletion

All three depletion runs continued until the aircraft's built-in failsafe engaged. Table 18 reports the threshold crossing, alert time, and time remaining between the alert and failsafe. For a depletion fault, the important measure is not how quickly the alert follows the crossing, but how much warning it provides before the aircraft responds automatically.

Table 18: Battery Depletion Runs

Run	Battery	Ambient (°C)	Onset (s)	Alert (s)	Failsafe (s)	Lead time (s)
FAULT-BAT-01	B1	21	526.0	533.0	664.8	138.8
FAULT-BAT-02	B2	21	330.0	336.0	452.2	122.2
FAULT-BAT-03	B2	20	323.0	329.0	451.4	128.4

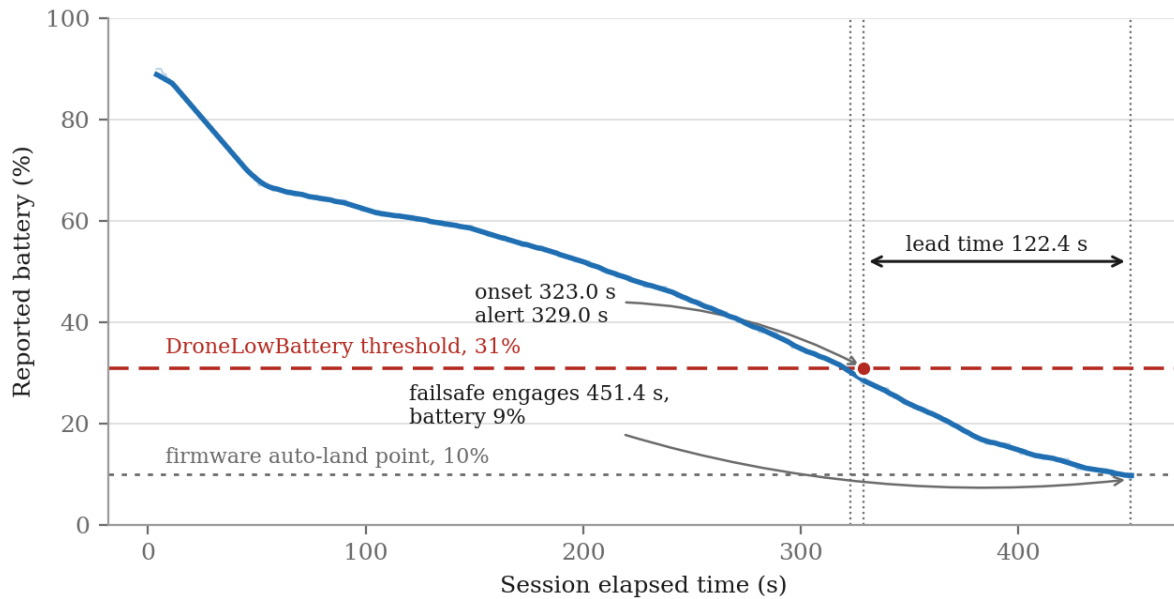


Figure 7: FAULT-BAT-03 depleting through the derived 31 % threshold. The alert fires 6 s after the crossing and leaves 122.4 s before the firmware failsafe engages at 9 %.

Figure 7 shows FAULT-BAT-03 in full, and it makes the shape of the result visible in a way the table cannot. The discharge is close to linear across the whole flight, so the crossing at 323.0 s is not a sudden event but the arrival of a trend that had been developing for minutes. The alert follows six seconds later, which at this scale sits almost on top of the crossing. What the figure adds is the size of the margin that remains. From the alert to the failsafe the aircraft still had 122.4 s of flight, and the battery fell from 31 to 9 % over that interval with no change in slope. The threshold therefore does not catch the aircraft at the edge of failure. It catches it at a point chosen so that the margin left over is long enough to act on, and the derivation in Section 5.5.3 is what set that distance.

The failsafe engaged with the reported battery level at 9 % in all three runs, which confirms the 10 % auto-land point the derivation assumed. Every run cleared the two-minute design target, and the shortest lead time exceeded it by 22 s. The spread between runs follows the depletion rate rather than anything about the rule: FAULT-BAT-01 used the slower battery and therefore crossed later in the flight and took longer to fall from the threshold to the failsafe point.

5.6.3 Thermal Stress

Each thermal run launched after a powered ground soak of approximately eight minutes. Table 19 gives the temperature at the first airborne sample, the in-flight range, and the matched healthy control WARM-01, which was flown on the same day under the same conditions without a soak.

Table 19: Thermal Runs and the Healthy Control

Run	Battery	Start bat. (%)	Launch temp (°C)	In-flight max (°C)	In-flight min (°C)	Alert (s)
FAULT-THERM-01	B1	57	75	75	64	10.0
FAULT-THERM-02	B2	84	86	86	65	11.0
FAULT-THERM-03	B2	48	90	90	65	10.0
WARM-01	B1	100	63	64	61	none

Figure 8 plots the hottest of the three fault runs against that control, and the two series never approach each other. The fault run begins 26 °C above the ceiling and falls steadily for the whole flight without returning to the healthy band, while the control stays between 61 and 64 °C throughout. That gap is the finding. The fault is not a rise that eventually crosses a threshold, it is a starting condition that flight slowly removes, which is the opposite of what the planned injection mechanism assumed. Both series are drawn as raw samples with a rolling mean over them, because the field is reported in whole degrees and the settled control oscillates by one degree, so the raw line alone would read as noise rather than as a steady operating point. The alert fires ten seconds into the flight, and the figure shows why that is not a detection delay in the ordinary sense: the aircraft was already far over the limit when it left the ground, so the only quantity being measured is the rule's own hold. The control reached exactly 64 °C at its warmest and never crossed the threshold, while the three fault runs launched at 11, 22 and 26 °C above it. In every fault run the maximum temperature is the launch temperature, and the aircraft cooled from that point toward the normal operating band for the remainder of the flight. This is the direct evidence for the claim in Section 5.3.2 that flight cools this aircraft.

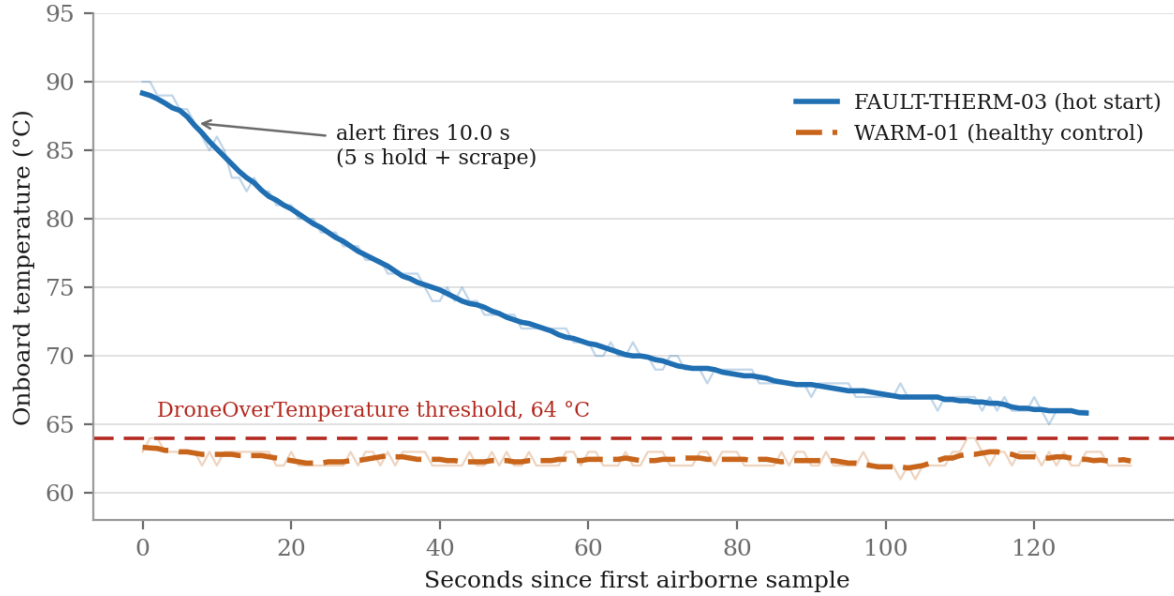


Figure 8: FAULT-THERM-03 against the healthy control WARM-01. Raw 1 Hz samples are shown faintly with an 11-sample rolling mean over them, because the field is reported in whole degrees. The fault run launches at 90 °C and cools towards the normal operating band, while the control never reaches the derived ceiling.

One consequence is that the thermal fault is present in the first airborne sample rather than developing during the flight. The latencies reported for these runs in Section 5.7.1 therefore measure detection at launch and are not comparable with the battery runs, where the fault developed over several minutes of flight.

5.6.4 Link Degradation

Each link run held a stationary hover until the ground station’s wireless interface was disabled. Table 20 reports the operator marker, the telemetry-derived onset and the alert.

Table 20: Link Degradation Runs

Run	Battery	Operator mark (s)	Onset (s)	Alert (s)	Latency (s)
FAULT-LINK-01	B1	78.8	152.0	156.0	4.0
FAULT-LINK-02	B1	88.4	94.0	97.0	3.0
FAULT-LINK-03	B1	109.7	116.0	119.0	3.0

FAULT-LINK-01 demonstrates why onset is taken from the telemetry rather than from the

operator marker. The marker sits 73.2 s before the telemetry-derived onset in that run, against 5.6 and 6.3 s in the other two. Had the marker been used as t_{onset} the run would have been reported with a detection latency of over a minute, which would have measured how long the operator took to complete the injection rather than how long the monitoring system took to notice it.

These runs also exposed a defect in the harness that is a result in its own right. The SDK serves state reads from a cached dictionary, so when the link drops the reads continue to return the last known state, the telemetry log remains complete at 1 Hz, and the poll-success flag stays at one. A liveness rule written against that flag would never have fired, and the monitoring stack would have reported a healthy aircraft for as long as the operator allowed the run to continue. The fix declares the link lost after three consecutive identical state vectors, which is sound because an airborne aircraft does not repeat its full state exactly. The latencies in Table 20 were measured with that fix in place. The SNR field behaves correctly once the link is gone, returning blank values with an honestly increasing staleness, which is why metric coverage for SNR falls to between 64 and 71 % on these three runs.

5.6.5 Detection Outcomes

Across the nine runs the rule matching the induced fault fired every time. Table 21 gives the confusion matrix. There were no false negatives and no misclassifications, where a misclassification would mean the intended fault went undetected while a different rule fired in its place.

Table 21: Detection Outcomes Across the Nine Fault Runs

Fault class	Runs	True positive	False negative	Misclassified
Battery depletion	3	3	0	0
Thermal stress	3	3	0	0
Link degradation	3	3	0	0
Total	9	9	0	0

A detection rate of nine of nine states only part of what happened, because it counts the intended fault and ignores everything else the rules did. Thirteen sustained alerts were raised across the nine runs, of which nine are the intended faults above and four were not intended. Table 22 lists them, because a monitoring system is judged by what it says as well as by what it catches.

Counting alert-level rather than run-level outcomes, ten of the thirteen sustained alerts correspond to a real condition on the aircraft, giving a precision of 0.77. The three genuinely

Table 22: Unintended Sustained Alerts Raised During the Fault Runs

Run	Alert	Onset	Assessment
FAULT-BAT-01	DroneOverTemperature	206.0 s	False positive. The aircraft was not hot by any operational standard, reaching 67 °C on an 11 min flight. The threshold was derived from four-minute flights, and the aircraft is still warming at four minutes
FAULT-BAT-02	DroneOverTemperature	255.0 s	False positive, same mechanism. Crossed at 65 °C on a 7.5 min flight
FAULT-THERM-01	DroneUnstableAttitude	99.0 s	False positive. Roll dispersion reached 4.29 against a threshold of 1.20 during the vertical oscillation the thermal mode flies. The baseline script contains no such manoeuvre, so the attitude threshold was never derived for it
FAULT-THERM-03	DroneLowBattery	115.0 s	Not a false positive. The run began on a 48 % cell, as the thermal mechanism requires, and the battery genuinely crossed 31 % in flight. The rule behaved correctly under a starting condition the protocol permits

false alerts share a single cause, and it is not the rule logic. In each case the alert fired in a flight regime that the derivation corpus does not contain: two on flights longer than any baseline, and one during a manoeuvre no baseline flies. A threshold derived from normal operation is only valid over the range of normal operation it was derived from, and each of these three alerts marks a point where the fault runs left that range.

5.7 Phase 3 Results: Evaluation (RQ2.3)

5.7.1 Detection Latency

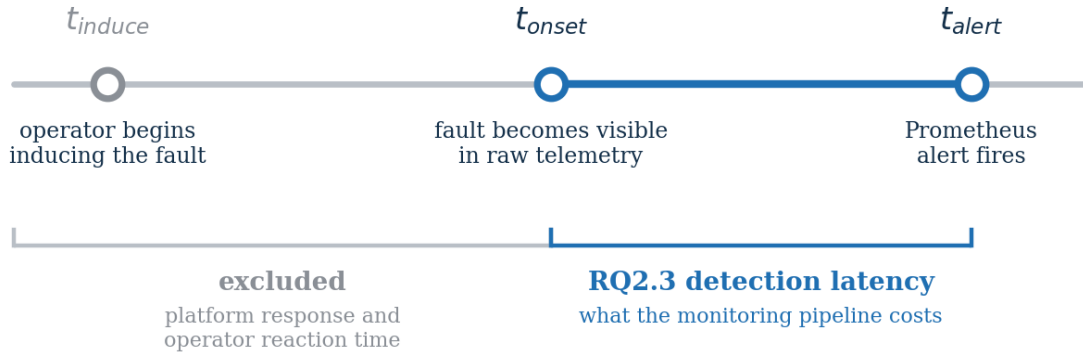


Figure 9: How detection latency is measured. Only the interval from the telemetry-derived onset to the Prometheus alert is counted.

Figure 9 defines what this measure includes, and the distinction decides the result. Three instants matter in a fault run. The operator records t_{induce} when they begin the injection, the telemetry crosses the threshold at t_{onset} , and Prometheus raises the alert at t_{alert} . Only the second interval is reported as detection latency. The first belongs to the operator and to the aircraft rather than to the monitoring system, so including it would make the result depend on how quickly the operator worked rather than on how quickly the rules responded. FAULT-LINK-01 shows how large that difference can be. Its operator marker sits 73.2 s before the telemetry onset, against 5.6 and 6.3 s in the other two link runs. Had the marker been used as the onset, that run would have been reported with a latency of over a minute, and almost all of the increase would have been measuring the operator rather than the pipeline.

Table 23 gives the latency measured for each detected fault, computed as $t_{\text{alert}} - t_{\text{onset}}$ from the Prometheus alert series and the telemetry log respectively.

Table 23: Detection Latency by Fault Class

Fault class	Rule	Hold	Latencies (s)	Mean (s)	n
Battery depletion	DroneLowBattery	5 s	7.0, 6.0, 6.0	6.3	3
Thermal stress	DroneOverTemperature	5 s	6.0, 7.0, 7.0	6.7	3
Link degradation	DronePollFailure	2 s	4.0, 3.0, 3.0	3.3	3

Across all nine detections the median latency is 6.0 s, the interquartile range is 3.5 to 7.0 s, and the maximum is 7.0 s.

The structure in Table 23 is the most useful result in this section. Every measured latency equals the rule's own hold plus one or two scrape intervals, and nothing else contributes. The two rules holding for 5 s detect in six to seven, and the rule holding for 2 s detects in three to four. The threshold value itself contributes nothing, because in each case the fault crosses the threshold decisively rather than approaching it gradually, so the crossing is unambiguous within a single scrape.

Detection speed on this platform is therefore set by the hold duration, not by the threshold. That is a design parameter rather than a property of the aircraft, and Section 5.6.1 establishes what shortening it would cost: the same healthy corpus that produced no alerts contains 33 crossings short enough to be suppressed by the current hold.

5.7.2 False Positive Rate

Across the three held-out healthy flights, the false positive rate was zero sustained alerts in 13.2 min of airborne time. This meets the target of no more than one sustained alert per 10 min. Including the five derivation sessions gives zero alerts across 35.1 min, but this figure is provided only for context. A 99th-percentile threshold is expected to be exceeded by around one percent of the data from which it was derived, so using that same data to measure the false positive rate would be circular.

The pre-induction portions of the fault runs are reported separately rather than included in this rate. Three of these segments produced the sustained alerts listed in Table 22, and pooling them with healthy flights would hide that result. The reported rate therefore applies only to four- to five-minute flights following the baseline manoeuvre script, which is the range covered by the healthy dataset. Section 5.6.5 shows that this result does not extend beyond that range.

5.7.3 Evaluation Criteria Results

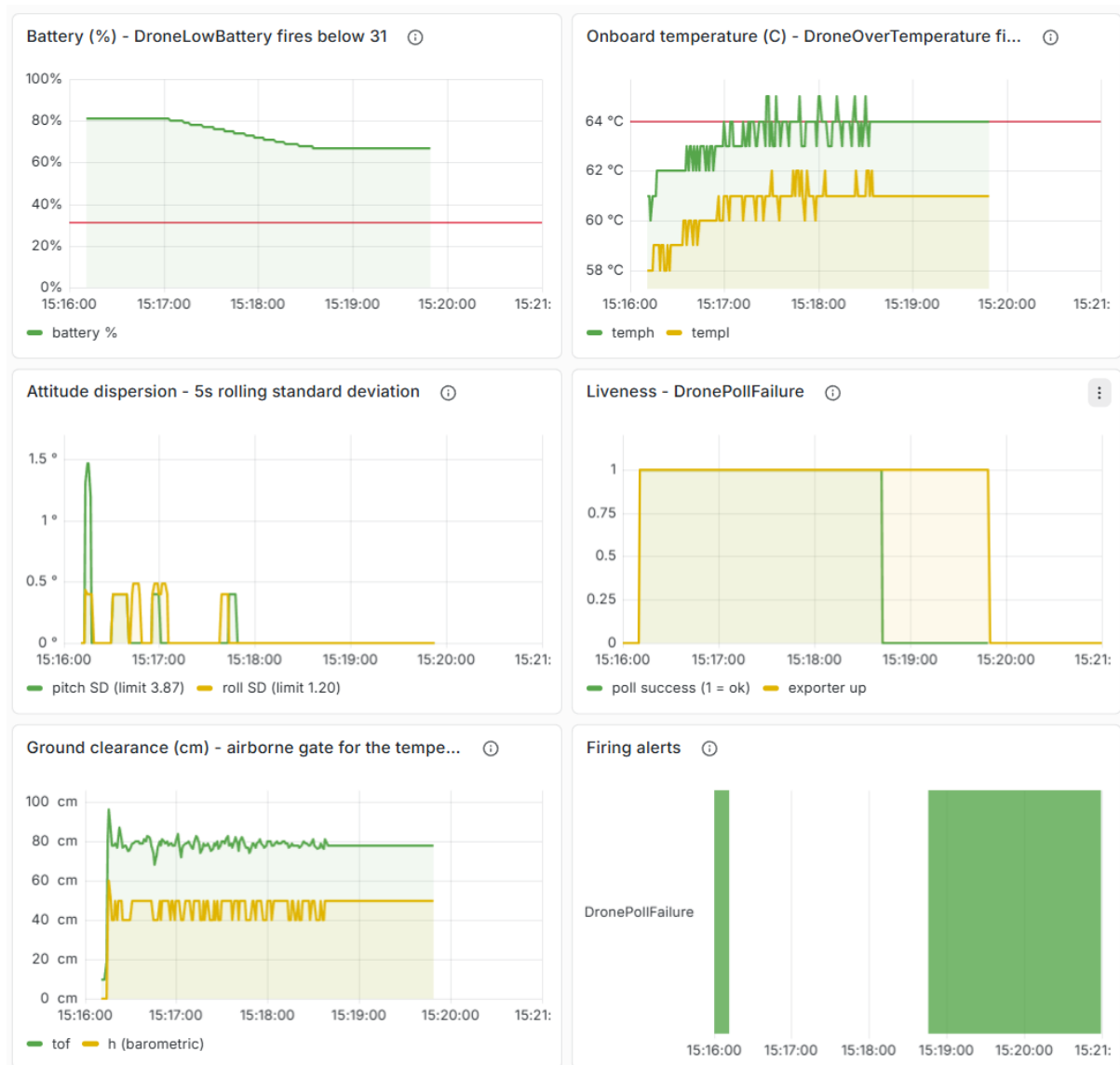


Figure 10: The Study 2 Grafana dashboard during FAULT-LINK-01, showing all four rule-bearing metrics, their derived thresholds and the live alert state on one screen.

Figure 10 is the evidence for the last of the six criteria, and it also shows the detection mechanism working end to end in a single view. All four rule-bearing metrics appear together, each drawn with the threshold derived for it in Phase 1. Battery sits well above the 31 % line for the whole flight and attitude dispersion stays far below its limit, so neither rule fires. Temperature oscillates around 62 °C and touches the 64 °C line several times without holding it, which is the suppression described in Section 5.6.1 visible as it happens. The liveness panel then

shows poll success dropping to zero partway through the flight, and the alert panel shows `DronePollFailure` firing immediately afterwards. Read together, those two panels are the result this study is about: a condition appears in a raw metric, and a rule derived from the aircraft's own baseline responds to it with no labelled data involved. The figure is drawn from stored data rather than captured during the flight, for the reason given in Section 5.3.5.

Table 24 summarises the outcome against each criterion fixed in Table 7.

Table 24: Study 2 Evaluation Criteria Results

Criterion	Target	Result	Met?
Threshold traceability	100% of rules	All four rules trace to a stated derivation rule applied to the Phase 1 corpus. No assumed value survives into the deployed rules file	Yes
Detection rate	≥ 8 of 9 runs	9 of 9. No false negatives and no misclassifications	Yes
Detection latency	Median ≤ 5 s; max ≤ 10 s	Median 6.0 s, maximum 7.0 s. The maximum target is met and the median is not, because latency equals the rule hold plus one or two scrape intervals and the hold is 5 s	Partial
False positive rate	≤ 1 per 10 min	Zero sustained alerts across 13.2 min of held-out normal flight. Valid for four to five minute flights of the baseline script; three sustained alerts occurred outside that envelope	Yes
Metric coverage	$\geq 99\%$ of samples	100 % for all four rule-bearing metrics in all eighteen accepted sessions. SNR carries no rule and is reported separately	Yes
Dashboard completeness	All metrics and alert state	Six panels showing battery, temperature, attitude dispersion, liveness, time-of-flight and the live alert state, each annotated with its derived threshold	Yes

Five of the six criteria were met in full and one was met in part. The partial result is the median latency, and it is a consequence of a design decision rather than a limitation of the platform. The 5 s hold was chosen to suppress transient crossings, it cannot be shortened without admitting some of the 33 crossings reported in Section 5.6.1, and a median latency of 6.0 s is the price of that choice. The target was set before the relationship between hold and latency was known.

5.8 Analysis

Study 2 starts from a simple claim: thresholds must be measured on the platform, not assumed in advance. The Phase 1 results support this claim regardless of the fault-run results. Of the three assumed thresholds taken into Study 2, one fired continuously during healthy operation and another could never fire. Neither was an unreasonable choice. Both were common values for the metric, but they did not fit this particular platform. Measuring the platform was the only way to show this.

The findings in Section 5.5.2 show the same issue in four fields from Table 5. Temperature rises towards a normal operating point instead of fluctuating around a stable average. Reported height drifts while the aircraft holds its true altitude. The barometric reference changes after every power cycle. Velocity provides no useful information when sampled at 1 Hz. A study using only replayed data would carry these assumptions forward, because the data cannot show that its own field labels are misleading.

There is also a wider finding about calibration. The revision branch in Step 4 of Figure 5 was used three times. Each time, the problem only appeared when a threshold was tested against data that differed from the data used to derive it.

1. **The wrong metric class.** Barometric height and temperature were both treated as bounded at first. The barometer re-zeroes on every power cycle, so the derived band described the power cycle and not the aircraft. Temperature climbs towards an operating point, so the derived ceiling sat below the range of normal flight.
2. **The wrong window.** Deriving the temperature threshold from the whole session gave 63 °C. The warm-up period pulled the value down, and a held-out flight then crossed it.
3. **The wrong flight length.** The corrected ceiling came from four-minute flights, but the aircraft is still warming at four minutes. Two longer battery runs, of seven and eleven minutes, crossed it while the aircraft was not hot. A third run, flown at 20 °C rather than 21 °C, peaked at exactly 64 °C and did not cross.

Section 5.6.5 adds a fourth case, and it is the clearest of the four because it involves a different metric. The attitude threshold came from a manoeuvre script with no vertical oscillation in it. The one fault run that flew such a profile raised a sustained attitude alert, even though the aircraft was flying exactly as commanded.

The pattern is the same every time. A derived threshold is only valid over the range of conditions it was measured across, and on this platform that range has at least four dimensions: the behaviour of the metric, the part of the session used, the length of the flight, and the manoeuvres flown.

This matters more than the choice of percentile. All four mistakes above would have survived any amount of care taken over that choice, because none of them was a question of picking the right number. Each was a question of whether the number described the right thing. Finding them took three steps: measuring the platform in normal operation, holding back further normal flights, and testing each threshold against data that played no part in producing it. Section 6 takes up what this means more widely and how it compares with the reviewed literature.

5.9 Answer to RQ2

RQ2 asked: *How can system metrics such as battery level, connectivity and performance be used to detect faults in remote autonomous systems?*

System metrics detect faults when each threshold is measured from the platform's own normal operation, derived by a rule chosen for how that metric behaves, held long enough to ignore brief spikes, and then tested against normal flights that took no part in setting it. Built this way, four rules derived from five baseline flights detected all nine induced faults across three fault classes, at a median latency of 6.0 s, with no sustained false alert in 13.2 min of held-out normal flight.

Each sub-question is answered below.

RQ2.1 asked which metrics represent operational health, and how their thresholds should be set. Four of the ten logged fields support a rule: battery percentage, onboard temperature, attitude dispersion and target liveness. These are this aircraft's expression of the four signal types named in Section 5.1, namely stored energy, thermal state, control stability and liveness, so the metric set is not particular to a quadrotor even though the fields carrying it are. Four fields do not support a rule, and in each case the reason is a property of the platform rather than of the method. Wireless SNR never changes, so no threshold can be derived from it. The barometric reference re-zeroes on each power cycle. Reported height drifts while the aircraft holds its altitude. The velocity fields read zero in 87.6 % of airborne samples. Thresholds are set by placing each metric in a behavioural class and applying the rule for that class, as set out in Figure 5.

RQ2.2 asked whether rules built this way detect induced faults. They detected nine of nine, against a target of eight, with no missed faults and no misclassifications. Thirteen sustained alerts were raised across those runs in total. Ten of them correspond to a real condition on the aircraft and three are false, and all three fired in flight conditions the baseline data does not cover.

RQ2.3 asked how the system performs on latency, false positives and coverage. The median latency is 6.0 s and the maximum is 7.0 s. In every case it equals the rule's hold plus one or two scrape intervals, so the hold is what controls detection speed. The false positive rate is zero per 10 min of held-out normal flight. Coverage is 100 % for every rule-bearing metric across all eighteen accepted sessions. Five of the six evaluation criteria were met in full, and the sixth, median latency, was met in part.

Three limits apply to these results, and the results themselves establish each one. The thresholds come from a single airframe, so they are not shown to transfer to another unit of the same model. They come from four-minute flights of one manoeuvre script, and Section 5.6.5 shows what happens outside that envelope. The nine fault runs cover three fault classes, chosen because they can be induced safely and repeatably indoors, which is not a representative sample of what a remote autonomous system meets in the field. Section 3.8 covers these limits in full.

6 Discussion

The two studies asked different questions of the same pipeline. Study 1 asked whether a monitoring framework can observe the health of a remote autonomous system, and answered yes using components that were already made rather than invented. Study 2 asked whether it can decide that health for itself. That turned out to be the harder question, and the difference between the two is where this work makes its contribution.

Study 1 met three of its four criteria with an exporter, Prometheus and Grafana, and its one partial result followed from a configuration value rather than from anything the framework could not do. What it could not do was decide what the telemetry meant, because its health state came from a label inside the dataset. The judgement had been made before the data reached the pipeline. Removing that label is what turns the exercise into a research problem, and Study 2 removed it. Two of the three thresholds carried in from ordinary practice then failed on contact with the platform. A ceiling of 60 °C sits below the temperature this aircraft reaches in normal flight, so it fired on every healthy session, and an attitude limit of 30° sits far above anything the aircraft does, so it could not fire at all. The issue was not that they were random values, what made them wrong was the platform they were applied to. The two failures are also not equally visible: one announces itself immediately, while the other is indistinguishable from a correct rule watching a healthy system until a fault occurs and nothing happens. Taken together, the studies show that the pipeline was never the hard part. Collecting telemetry and putting it on a screen is solved by existing tooling. Deciding what it means for a particular platform is not, and that is where the risk sits.

Deriving thresholds from measured data is necessary but not sufficient. The evidence is that all four calibration errors reported in Section 5.8 happened after the decision to derive empirically had already been taken, and each came from real measurements of the real aircraft. They share a structure: in every case the data used to derive the threshold failed to represent a condition the rule would later meet. A derived threshold is therefore not a number. It is a number together with the range of conditions it was measured across, and reporting just a number without the actual testing leaves a reader unable to judge whether it applies to their case. Finding such a mismatch also requires held-out data of a different shape rather than more of the same. Three held-out flights of the same length and script exposed nothing, while the first flight of a different length exposed the temperature ceiling at once. One parameter sits outside this and should not be confused with it. Detection latency equalled each rule's hold plus one or two scrape intervals in every run, so calibration decides whether a fault is detected and the hold decides how quickly.

The harness defect reported in Section 5.6.4 carries the same lesson in a different form. After the radio link dropped, the SDK continued to serve state reads from a cache, so the telemetry log stayed complete at 1 Hz, the poll-success flag stayed at one, and every field held a value that had been plausible a moment earlier. The monitoring system did not report an error. It reported perfect health. Liveness cannot be confirmed simply because no error is reported. No error may also mean the system is no longer reporting anything at all, and any recovery logic based on that signal will fail silently too.

Compared with the reviewed literature, this shifts the focus slightly. Threshold-based detection research tends to focus on choosing thresholds and calculating them (Yousuf et al. 2024). None of the four errors found here came from a calculation error. They came from assumptions about the platform and its data. The machine-learning approaches in Section 2.4 avoid hand-set thresholds, but not this underlying problem. A model can only recognise what is represented in its training data, just as a percentile threshold is limited by the data used to derive it. The Prometheus architecture described by Boncea & Bacivarov (2016) was suitable for a physically mobile platform that operates offline. The `for` clause had the greatest effect on detection latency. This is the Prometheus setting that requires an alert condition to remain true for a set time before the alert fires. Its hold duration mattered more than the scrape interval.

What these claims rest on should be stated as plainly as the thresholds themselves. The calibration procedure is expressed in terms of how metrics behave, but it was applied on one platform, and the measured values come from a single airframe, one indoor venue, a narrow ambient range and one manoeuvre script. The three fault classes were chosen because they can be induced safely and repeatably indoors, and all three cross their thresholds sharply, which is what makes latency equal the hold. The central finding of this work is that thresholds do not transfer beyond the conditions they were measured in, and that applies to these results as much as to any others. Section 3.8 sets out the experimental threats in full.

7 Conclusion

Remote autonomous systems operate where failure is costly and human intervention is slow, yet most monitoring approaches available to them are reactive and were developed for static industrial systems. This thesis investigated whether an open-source monitoring framework can assess the health of such a system, and whether it can detect faults in one without being told in advance what a fault looks like.

RQ1 was answered by building the pipeline and validating it against replayed drone telemetry. A Python exporter, Prometheus and Grafana collected, stored, evaluated and displayed health telemetry, identified every labelled fault transition with no false positives, and met three of four evaluation criteria in full. The single partial result, detection latency, followed from the scrape interval, which is a configuration value rather than a limitation of the framework. That study established the architecture, but the health state it reported came from a label inside the dataset rather than from the telemetry itself.

RQ2 removed that dependency by connecting the same pipeline to a physical DJI Tello. Four alerting rules were derived from five baseline flights, each threshold following from a stated rule applied to the aircraft's own measured data. Across nine induced faults in three classes the rules detected nine of nine, at a median latency of 6.0 s, with no sustained false alert in 13.2 min of held-out normal flight. Five of the six evaluation criteria were met in full. Thirteen sustained alerts were raised in total, of which three were false, and all three fired in flight conditions the baseline data did not cover.

That last point is the main contribution of this work. Deriving thresholds from measured data is necessary but not sufficient, because a threshold is only valid across the range of conditions it was measured in. Calibration scope was found to be wrong four times in this study: by metric class, by the window within a flight, by flight duration, and by manoeuvre profile. Each error was invisible until held-out data of a different shape met it. The practical consequence is that every derived threshold should state the conditions it was derived from, and be tested against healthy data with a different profile, not simply more of the same data.

For specialists in the field, this work provides a reusable approach to monitoring a remote autonomous system using open-source tools, without cloud infrastructure or labelled training data. For researchers, it evaluates Prometheus directly on a physical autonomous system, measuring detection latency and false positive rate rather than assuming them.

7.1 Future Work

The most direct extension is to act on a detected fault rather than only report it, which would close the self-healing half of the gap identified in Section 2.8. The harness defect reported in Section 5.6.4 should be addressed first, because recovery logic built on a liveness signal that can fail silently would inherit that silence.

Three further directions follow from the limits of this study. Repeating the baseline flights on the second identical airframe would test whether the derived thresholds transfer beyond a single unit. Collecting longer healthy flights with ambient temperature recorded alongside the flight data would extend the temperature ceiling beyond its current four-to five-minute range. Applying the same calibration procedure to a ground or underwater platform would test whether the four behavioural classes hold for remote autonomous systems generally, or only for a small quadrotor.

References

- Alauthman, A. & Al-Hyari, A. (2025), 'Intelligent fault detection and self-healing mechanisms in wireless sensor networks using machine learning and Flying Fox Optimization', *Computers* **14**(6), 233.
- Barile, G., Leoni, A., Pantoli, L. & Stornelli, V. (2018), 'Real-time autonomous system for structural and environmental monitoring of dynamic events', *Electronics* **7**(12), 420.
- Barrett, H., Matthews, J., Ford, A. & Castro, H. (2023), 'Observability and monitoring using Prometheus and Grafana in cloud setups', ResearchGate. Accessed 13 May 2026.
URL: <https://www.researchgate.net/publication/392163369>
- Ben Hafaiedh, I. & Ben Slimane, M. (2022), 'A distributed formal-based model for self-healing behaviors in autonomous systems: from failure detection to self-recovery', *Journal of Supercomputing* **78**, 18725–18753.
- Boncea, R. & Bacivarov, I. (2016), A system architecture for monitoring the reliability of IoT, in 'Proceedings of the 15th International Conference on Quality and Dependability', IEEE.
- DeMedeiros, K., Hendawi, A. & Alvarez, M. (2023), 'A survey of AI-based anomaly detection in IoT and sensor networks', *Sensors* **23**(3), 1352.
- Devi, K., Thenmozhi, R. & Kumar, D. S. (2024), Self-healing IoT sensor networks with isolation forest algorithm for autonomous fault detection and recovery, in 'Proceedings of the 2024 International Conference on Automation, Computation and Control (AUTOCOM)', pp. 451–456.
- Ekanayakadevlk, R. (2024), 'Drone telemetry tampering dataset v2', <https://www.kaggle.com/datasets/rasikaekanayakadevlk/drone-telemetry-tampering-dataset-v2>. Accessed: May 2026. Licence: CC BY-SA 4.0. UAV telemetry dataset for tampering detection and anomaly analysis.
- Ghafouri, A., Abbas, W., Laszka, A., Vorobeychik, Y. & Koutsoukos, X. (2016), Optimal thresholds for anomaly-based intrusion detection in dynamical environments, in Q. Zhu, T. Alpcan, E. Panaousis, M. Tambe & W. Casey, eds, 'Decision and Game Theory for Security: 7th International Conference, GameSec 2016', LNCS 9996, Springer, pp. 415–434.
- Gültekin, Ö., Cinar, E., Özkan, K. & Yazıcı, A. (2022), 'Real-time fault detection and condition monitoring for industrial autonomous transfer vehicles utilizing edge artificial intelligence', *Sensors* **22**(9), 3208.
- Hemalatha, S., Reddy, K. V. S. V. T., Rao, T. V., Ramaswamy, T., Pillai, N. M. & Mohan, G. K. (2025), 'Advancing autonomous systems: A review of emerging trends in robotics', *Journal Européen des Systèmes Automatisés* **58**(5), 913–921.
- Hevner, A. R., March, S. T., Park, J. & Ram, S. (2004), 'Design science in information systems research', *MIS Quarterly* **28**(1), 75–105.

- Mazare, M., Taghizadeh, M., Ghaf-Ghanbari, P. & Davoodi, E. (2024), 'Robust fault detection and adaptive fixed-time fault-tolerant control for quadrotor UAVs', *Robotics and Autonomous Systems* **179**, 104747.
- Ogunmolu, A. M., Olaniyi, O. O., Popoola, A. D., Olisa, A. O. & Bamigbade, O. (2025), 'Autonomous artificial intelligence agents for fault detection and self-healing in smart manufacturing systems', *Journal of Energy Research and Reviews* **17**(8), 20–37.
- Paradeshi, K. P., Shaikh, J. A. & Sayyad Liyakat, K. K. (2022), 'Implementation of fault detection framework for healthcare monitoring system using iot, sensors in wireless environment', *Telematique* **21**(1), 5451–5460.
- Puchalski, R. & Giernacki, W. (2022), 'UAV fault detection methods, state-of-the-art', *Drones* **6**(11), 330.
- Rafiq, A., Shakir, M. Z., Gray, D., Inglis, J. & Ferguson, F. (2025), 'AI and IoT-driven monitoring and visualisation for optimising MSP operations in multi-tenant networks: a modular approach using sensor data integration', *Sensors* **25**(19), 6248.
- Rafique, S. H., Abdallah, A., Musa, N. S. & Murugan, T. (2024), 'Machine learning and deep learning techniques for Internet of Things network anomaly detection: current research trends', *Sensors* **24**(6), 1968.
- Reichard, K., Crow, E. & Rogan, C. (2007), Integrated system health management in unmanned and autonomous systems, in 'AIAA Infotech@Aerospace 2007 Conference and Exhibit', Infotech@Aerospace Conferences, American Institute of Aeronautics and Astronautics.
- Reis, M. J. C. S. & Reis, A. J. D. (2025), 'Edge-based real-time fault detection in uav systems via b-spline telemetry reconstruction and lightweight hybrid ai', *Sensors* **25**(16), 4944.
- Singh, K., Yadav, M., Singh, Y. & Moreira, F. (2025), 'Techniques in reliability of Internet of Things', *Procedia Computer Science* **256**, 55–62.
- Smallman, H. S. & Cook, M. B. (2013), Proactive supervisory decision support from trend-based monitoring of autonomous and automated systems: A tale of two domains, in R. Shumaker, ed., 'Virtual Augmented and Mixed Reality (VAMR/HCI 2013), Part II', Vol. 8022 of *Lecture Notes in Computer Science*, Springer-Verlag Berlin Heidelberg, pp. 320–329.
- Wang, L., Cheng, Y., Jiang, B., Zhang, Y., Zhu, J. & Tan, X. (2025), 'Adaptability study of an unmanned aerial vehicle actuator fault detection model for different task scenarios', *Drones* **9**(5), 360.
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B. & Wesslén, A. (2012), *Experimentation in Software Engineering*, Springer.
- Yousuf, M., Alsuwian, T., Amin, A., Fareed, S. & Hamza, M. (2024), 'IoT-based health monitoring and fault detection of industrial AC induction motor for predictive maintenance', *Measurement and Control*.